GUC
German University in Cairo

CSIS 402 : COMPUTER ORGANIZATION & SYSTEM PROGRAMMING

JUMPS - LOOPS

LECTURE 8

Dr. Sarah Azzam

Slides Based on Lecture Slides by Assoc. Prof. Dr. Amr Talaat

INTERVIEW QUESTIONS ON INTEL 8086

- What is data and address size in 8086?
- Write the flags in 8086?
- What is the function of BIU?
- What is the function of EU?
- What are the conditions under which the EU enter into the WAIT mode?
- Advantages of memory segmentation in 8086
- Characteristics of the CMP instructions. - 8086

ADDITIONAL QUESTIONS

Q1. How does the 8086 differ from the 8085 microprocessor?
Q2. Point out the differences between the 8086 and 8088 microprocessor?
Q2. How many bit microprocessor is the 8086 microprocessor?
Q3. Explain the internal architecture of the 8086 microprocessor?
Q4. What is the purpose of the BIU in the 8086?
Q5. What does the EU do in the 8086?
Q6. State the number and type of registers in the 8086?
Q7. What is the purpose of segment register?
Q8. What are the index registers in 8086?
Q9. Define the number of flags present in the 8086 and name each one of them.
Q10. What does the parity flag do?
Q11. What is single stepping and how can it be achieved on the 8086?
Q12. With the help of an example explain how physical address is calculated?

INSTRUCTION TYPES

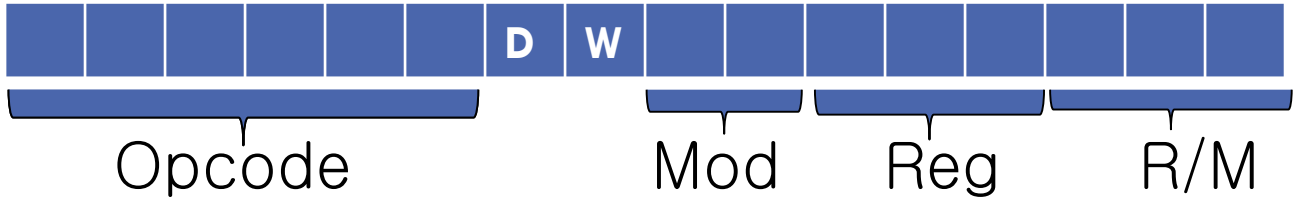
- Data Movement
- Arithmetic/Logic ✓
- Program Control (if,..)
- Loop Control
- Interrupts

THE OPCODE

Opcode						D	W
1	0	0	0	1	0	1	1

MOD		REG			R/M		
1	1	1	0	1	1	0	0

Opcode = MOV
D = Transfer to register (REG)
W = Word
MOD = R/M is a register
REG = BP
R/M = SP



MOV BP, SP

Field	Description
D	the direction of data flow either from or to R/M from REG. If D = 0 REG → R/M and if D = 1 R/M → REG
W	specifies a word (16-bit mode) or a doubleword (32-bit mode). W = 0 byte and W = 1 word/doubleword
MOD	specifies how R/M is used and whether a displacement exists. MOD = 00 memory no displacement MOD = 01 memory 8-bit displacement MOD = 10 memory 16/32-bit displacement MOD = 11 register

REG	W=0	W=1
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SI
111	BH	DI

MOD=11		
R/M	W=0	W=1
000	AL	AX
001	CL	CX
010	DL	DX
011	BL	BX
100	AH	SP
101	CH	BP
110	DH	SI
111	BH	DI

Program …*a sample*

```
;THE FORM OF AN ASSEMBLY LANGUAGE PROGRAM
;NOTE: USING SIMPLIFIED SEGMENT DEFINITION
        .MODEL SMALL
        .STACK 64
        .DATA
DATA1           DB      52H
DATA2           DB      29H
SUM             DB      ?
        .CODE
MAIN            PROC    FAR         ;this is the program entry point
        MOV     AX,@DATA            ;load the data segment address
        MOV     DS,AX               ;assign value to DS
        MOV     AL,DATA1            ;get the first operand
        MOV     BL,DATA2            ;get the second operand
        ADD     AL,BL               ;add the operands
        MOV     SUM,AL  ;store the result in location SUM
        MOV     AH,4CH  ;set up to
        INT     21H                 ;return to DOS
MAIN            ENDP
        END     MAIN                ;this is the program exit point
```

SEGMENTS

Segment definition:

The 80x86 CPU has four segment registers: CS, DS, SS, ES

Segments of a program:

.STACK ; marks the beginning of the stack segment

example:

```
.STACK  64      ;reserves 64B of memory for the stack
```

.DATA ; marks the beginning of the data segment

SEGMENTS

.CODE ; marks the beginning of the code segment

- starts with PROC (procedures) directive
- the PROC directive may have the option FAR or NEAR
- ends by ENDP directives

ASSEMBLER DATA DIRECTIVES

- **ORG** (origin) – to indicate the beginning of the offset address
  - *example:*

```
        ORG     0010H
```

- **DB** (define byte) – allocation of memory in byte-sized chunks
  - *example:*

```
DATA1   DB  25          ;decimal
DATA2   DB  10001001B   ;binary
DATA3   DB  12H         ;hex
DATA4   DB  '2591'          ;ASCII numbers
DATA5   DB  ?           ;set aside a byte
DATA6   DB  "O' Hi"         ;ASCII characters
```

ASSEMBLER DATA DIRECTIVES

- **DUP** (duplicate) – to duplicate a given number of characters
  - *example:*

```
DATA1     DB      0FFH, 0FFH, 0FFH, 0FFH        ;fill 4 bytes with FF
```

Can be replaced with:

```
DATA2     DB      4 DUP(0FFH)                   ;fill 4 bytes with FF

DATA3     DB      30 DUP(?)                     ;set aside 30 bytes

DATA4     DB      5 DUP (2 DUP (99))            ;fill 10 bytes with 99
```

DATA TYPES AND DEFINITIONS

- **DW** (define word) – allocate memory 2 bytes (one word) at a time

example:

```
DATA1    DW    342              ;decimal
DATA2    DW    01010001001B     ;binary
DATA3    DW    123FH            ;hex
DATA4    DW    8 DUP (?)        ;set aside 8 words
```

DATA TYPES AND DEFINITIONS

- **DD** (define doubleword) – allocate memory 4 bytes (2 words) at a time
  - *example:*

```
DATA1    DD    1023                ;decimal
DATA2    DD    01010001001001110110B   ;binary
DATA3    DD    7A3D43F1H           ;hex
```

- **DQ** (define quadwordequate) – allocate memory 8 bytes (4 words) at a time
  - *example:*

```
DATA1    DQ    6723F9H ;hex
DATA2    DQ    'Hi'    ;ASCII characters
DATA3    DQ    ?       ;nothing
```

THE JUMP GROUP

- Allows programmer to skip program sections and branch to any part of memory for the next instruction.
- A **conditional jump** instruction allows decisions based upon numerical tests.
  - results are held in the flag bits, then tested by conditional jump instructions
- **LOOP** and **conditional LOOP** are also forms of the jump instruction.

UNCONDITIONAL JUMP (JMP)

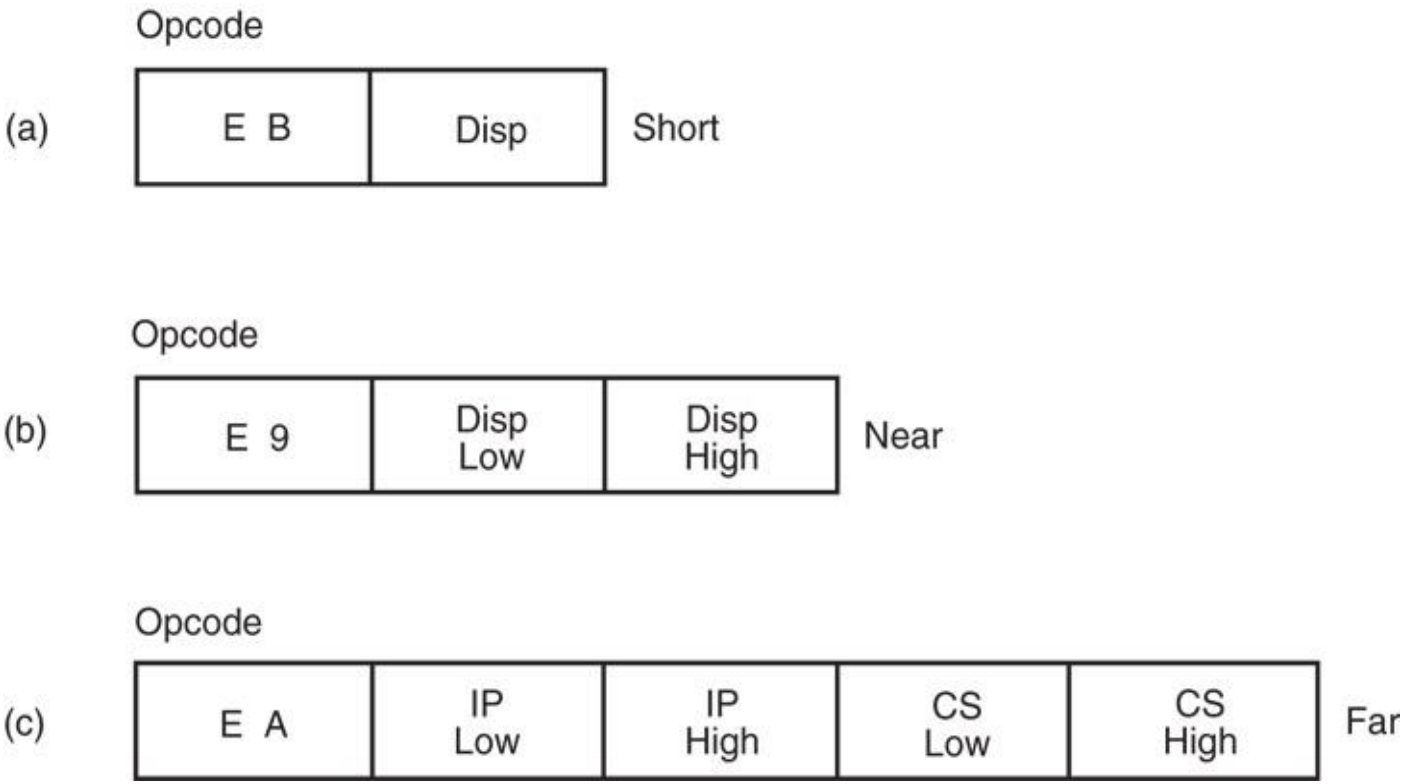
- Three types: **short** jump, **near** jump, **far** jump.
- **Short jump** is a 2-byte instruction that allows jumps or branches to memory locations within **+127** and **–128** bytes.
  - From the address **following** the jump
- 3-byte **near jump** allows a branch or jump within ±32K bytes from the instruction in the current code segment.

UNCONDITIONAL JUMP (JMP)

- 5-byte **far jump** allows a jump to any memory location within the real memory system.
- The **short** and **near** jumps are often called **intra*segment* jumps.**
- **Far jumps** are called **inter*segment* jumps**.

UNCONDITIONAL JUMP (JMP)

The three main forms of the JMP instruction. Note that Displacement is either an **8-** or **16-bit** signed **displacement** or distance.

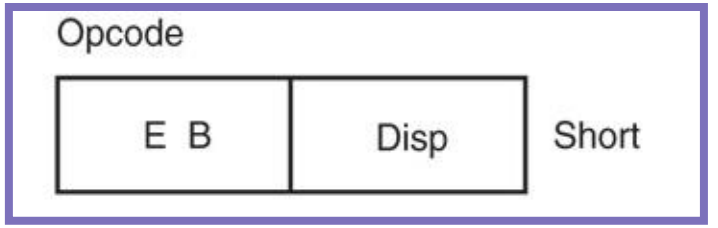


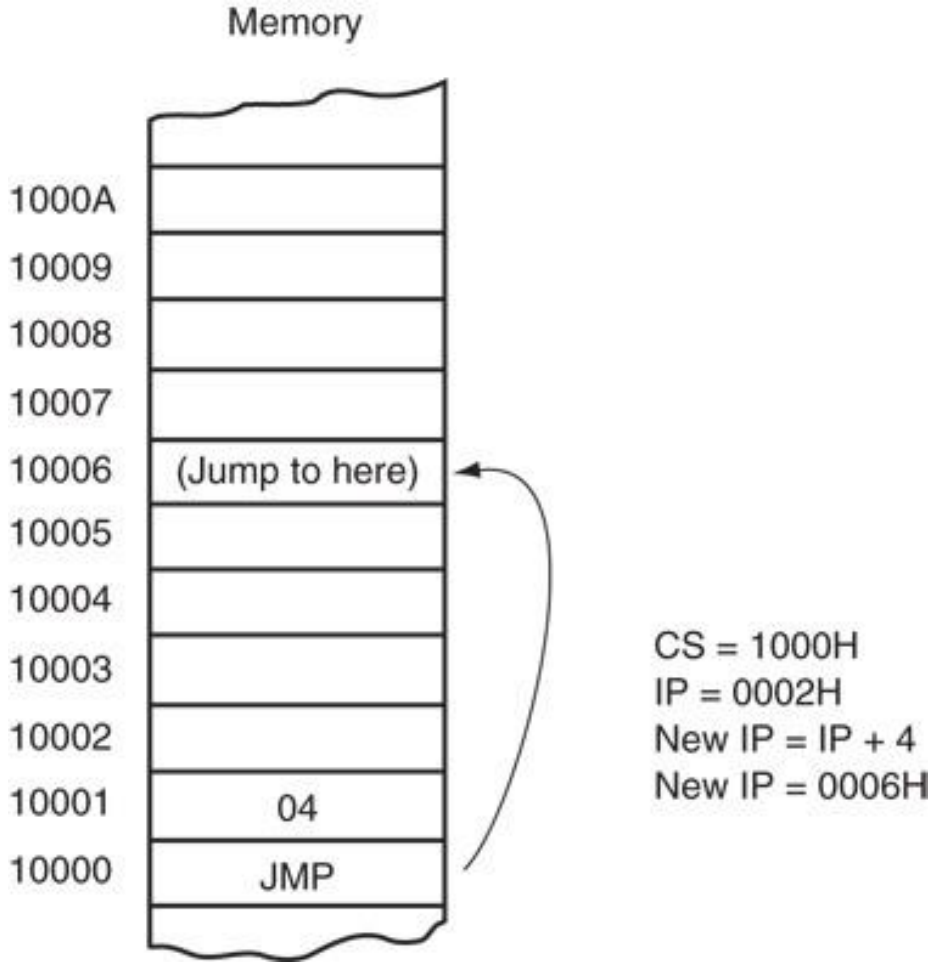
SHORT JUMP

When the microprocessor executes a **short jump**, the displacement is sign-extended and added to the instruction pointer (IP/EIP) to generate the jump address within the current code segment

The instruction branches to this new address for the next instruction in the program.

A short jump to four memory locations beyond the address of the next instruction.



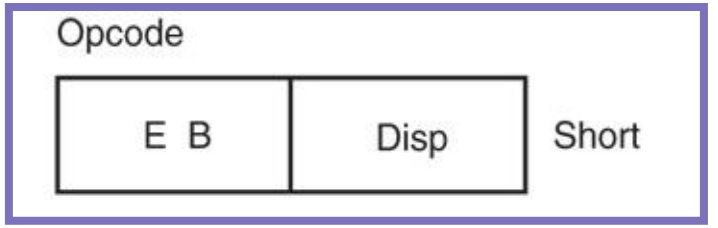


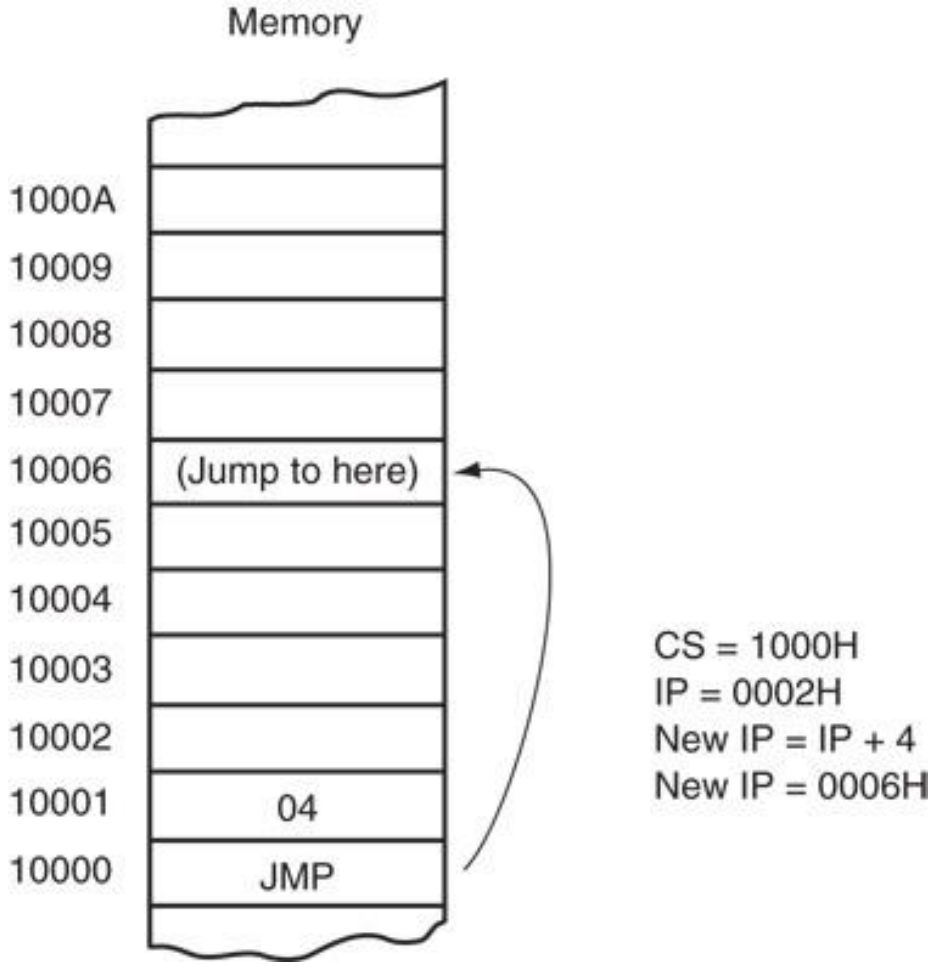
SHORT JUMP

When the microprocessor executes a **short jump**, the displacement is sign-extended and added to the instruction pointer (IP/EIP) to generate the jump address within the current code segment

The instruction branches to this new address for the next instruction in the program.

A short jump to four memory locations beyond the address of the next instruction.





JUMPS

- When a jump references an address, a **label** normally identifies the address.
- The **JMP NEXT** instruction is an example.
  - it jumps to **label NEXT** for the next instruction
  - very rare to use an actual hexadecimal address with any jump instruction
- The label NEXT must be followed by a colon (**NEXT:**) to allow an instruction to reference it.
  - if a colon does not follow, you cannot jump to it
- The only time a colon is used is when the label is used with a jump or call instruction.

NEAR JUMP

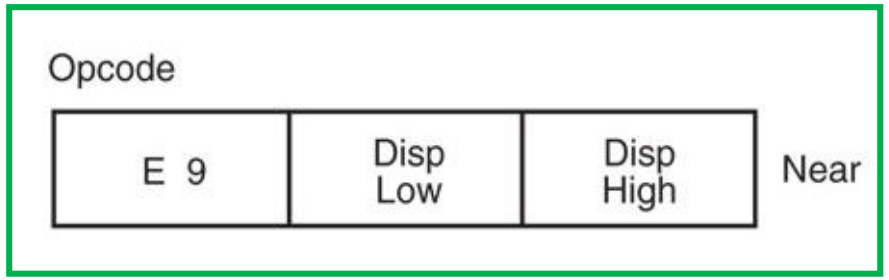
- A **near jump** passes control to an instruction in the current code segment located within **±32K bytes** from the **near jump** instruction.
  - distance is ±2G in 80386 and above when operated in protected mode .
- **Near jump** is a 3-byte instruction with opcode followed by a signed 16-bit displacement.
  - 80386 - Pentium 4 displacement is 32 bits and the near jump is 5 bytes long.

NEAR JUMP

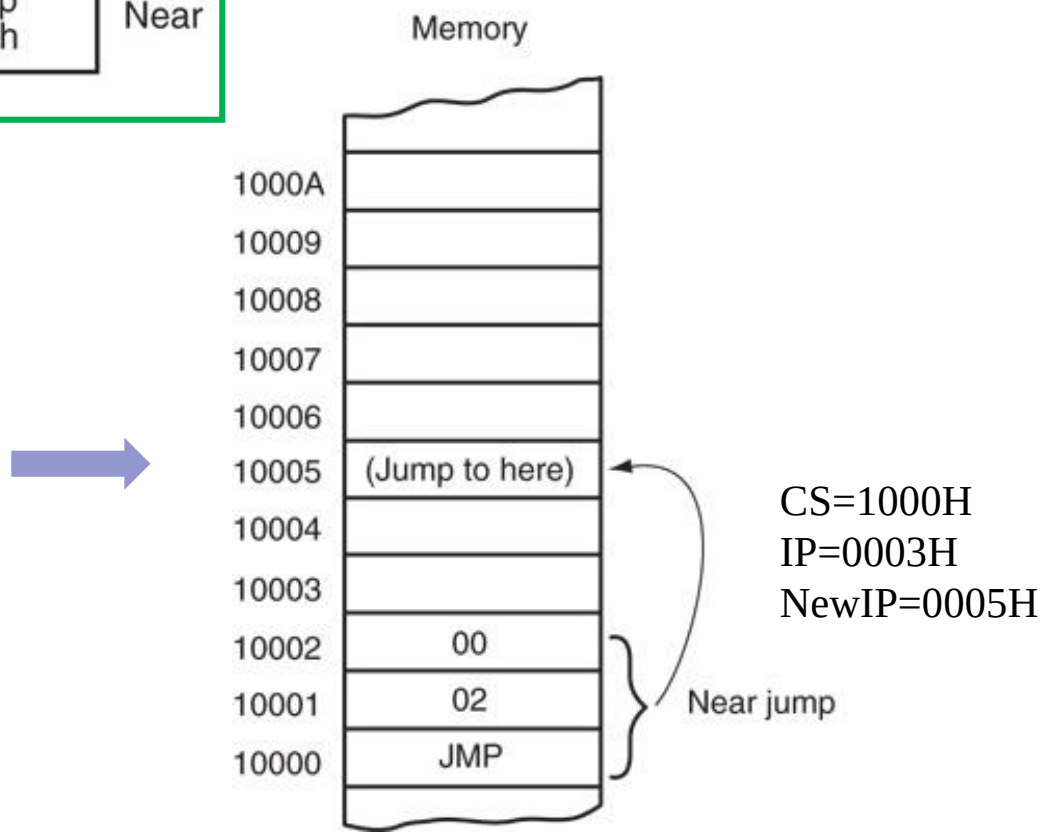
- Signed displacement adds to the instruction pointer (IP) to generate the jump address.
  - because signed displacement is ±32K, a near jump can jump to any memory location within the current code segment.

NEAR JUMP

EXAMPLE



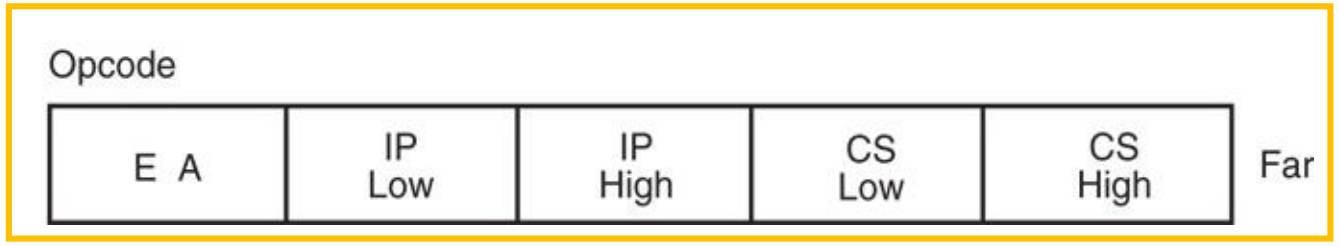
The instruction branches to this new address for the next instruction in the program.



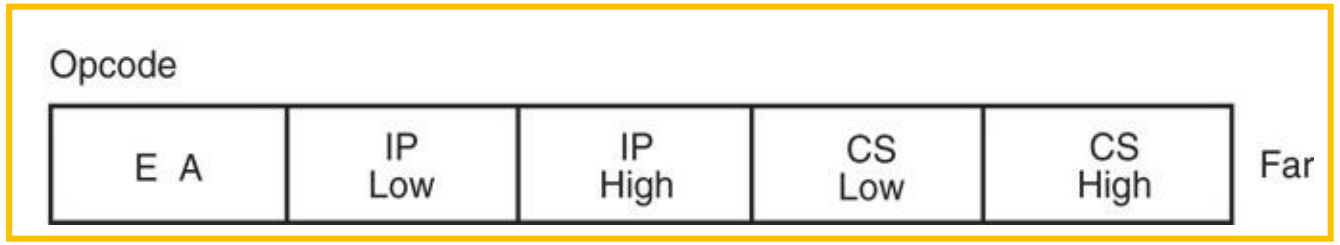
A near jump that adds the displacement (0002H) to the contents of IP

FAR JUMP

- Obtains a new segment and offset address to accomplish the jump:
  - bytes 2 and 3 of this 5-byte instruction contain the new offset address
  - bytes 4 and 5 contain the new segment address

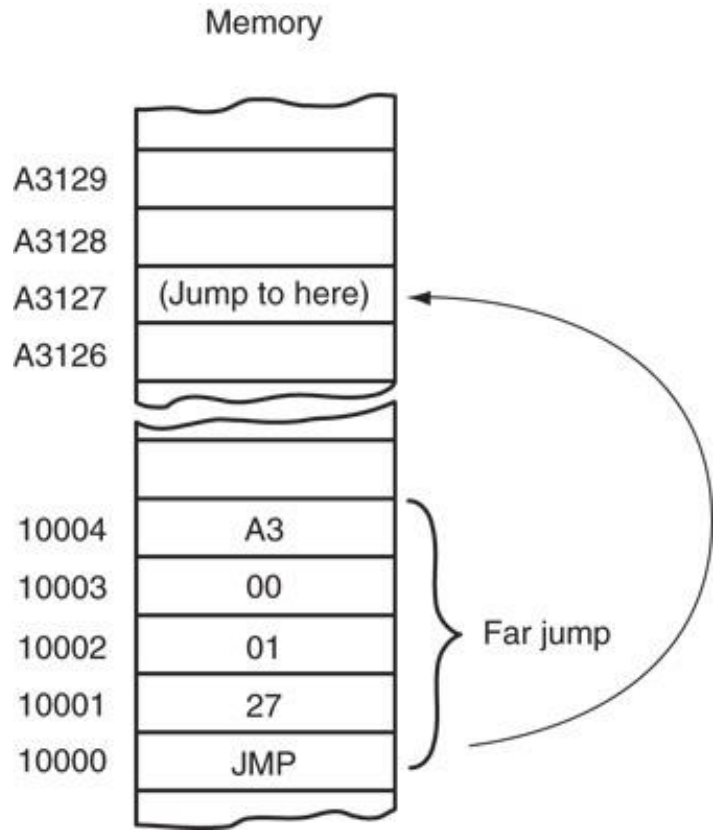


FAR JUMP



EXAMPLE

The instruction branches to this new address for the next instruction in the program.



A far jump instruction replaces the contents of both CS and IP with 4 bytes following the opcode.

INSTRUCTION TYPES

- Data Movement
- Arithmetic/Logic
- Program Control (if,..)
- Loop Control
- Interrupts

REGISTER INDIRECT JUMP

- Jump can also use a 16- or 32-bit register as an operand.
  - The instruction is automatically set up as an ***indirect jump***.
  - Address of the jump is in the register specified by the jump instruction
- Unlike displacement associated with the near jump, **register contents** a re transferred directly into the **instruction pointer**.
- An ***indirect jump*** does not add to the instruction pointer.

REGISTER INDIRECT JUMP

EXAMPLE

JMP AX

- copies the contents of the AX register into the IP.
- allows a jump to any location within the current code segment.

- In 80386 and above, JMP EAX also jumps to any location within the current code segment.

MEMORY INDIRECT JUMP

- The target address is the two memory locations pointed at by the register.

EXAMPLE

JMP [DI]

- The IP will be replaced by the contents of memory locations pointed at by DS:DI and DS:DI+1.

CONDITIONAL JUMPS

- Always short jumps in 8086 - 80286.
  - limits range to within +127 and –128 bytes from the location following the c onditional jump
- In 80386 and above, conditional jumps are either short or near jumps (±32K)
- Conditional jump instructions test flag bits:
  - sign (S), zero (Z), carry (C)
  - parity (P), overflow (0)

Mnemonic	Condition Tested	"Jump IF ..."
JA/JNBE	(CF = 0) and (ZF = 0)	above/not below nor zero
JAE/JNB	CF = 0	above or equal/not below
JB/JNAE	CF = 1	below/not above nor equal
JBE/JNA	(CF or ZF) = 1	below or equal/not above
JC	CF = 1	carry
JE/JZ	ZF = 1	equal/zero
JG/JNLE	((SF xor OF) or ZF) = 0	greater/not less nor equal
JGE/JNL	(SF xor OF) = 0	greater or equal/not less
JL/JNGE	(SF xor OR) = 1	less/not greater nor equal
JLE/JNG	((SF xor OF) or ZF) = 1	less or equal/not greater
JNC	CF = 0	not carry
JNE/JNZ	ZF = 0	not equal/not zero
JNO	OF = 0	not overflow
JNP/JPO	PF = 0	not parity/parity odd
JNS	SF = 0	not sign
JO	OF = 1	overflow
JP/JPE	PF = 1	parity/parity equal
JS	SF = 1	sign

CONDITIONAL JUMPS

- If the condition under test is true, a branch to the label associated with the jump instruction occurs.
  - if false, next sequential step in program executes

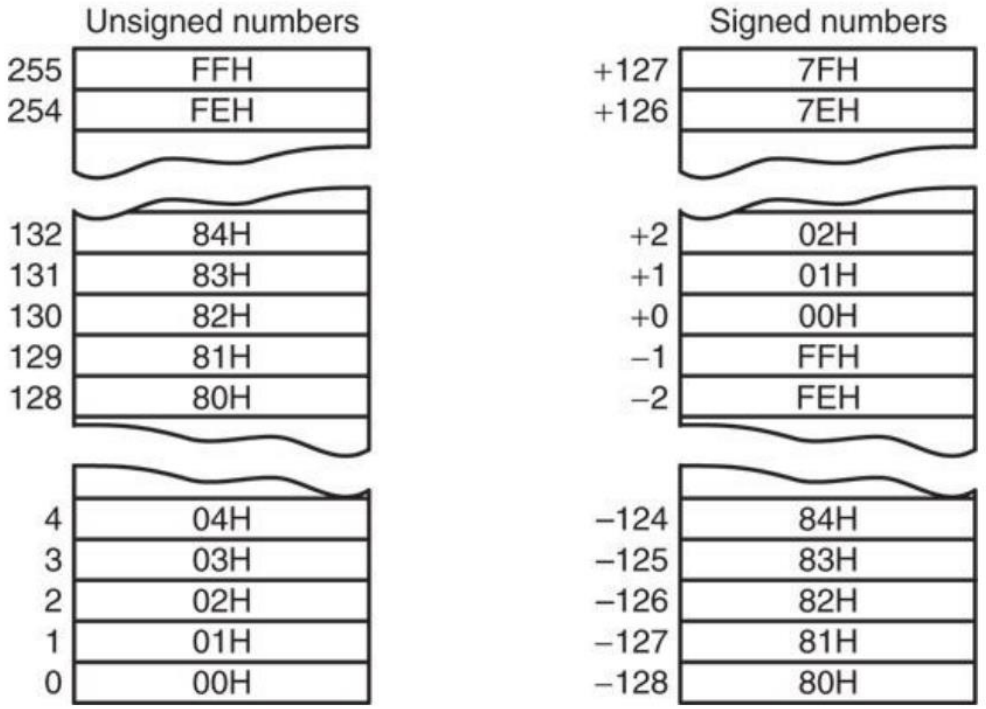


a **JC** will jump if the carry bit is set

- Most conditional jump instructions are straightforward as they often test one flag bit. However, some test more than one.

CONDITIONAL JUMPS

- Because both signed and unsigned numbers are used in programming
- 16- and 32-bit numbers follow the same order as 8-bit numbers, except that they are larger.



Signed and unsigned 8-bit numbers

CONDITIONAL JUMPS

- When signed numbers are compared, use the

  JG, JL, JGE, JLE, JE, and **JNE** instructions.

  - terms greater than and less than refer to signed numbers

- When unsigned numbers are compared, use the

  JA, JB, JAB, JBE, JE, and **JNE** instructions.

  - terms above and below refer to unsigned numbers

CONDITIONAL JUMPS

- Remaining conditional jumps test individual flag bits, such as overflow and parity.
  - notice that JE has an alternative op-code JZ

- All instructions have alternates, but many aren’t used in programming because they don’t usually fit the condition under test.

GUC
German University in Cairo

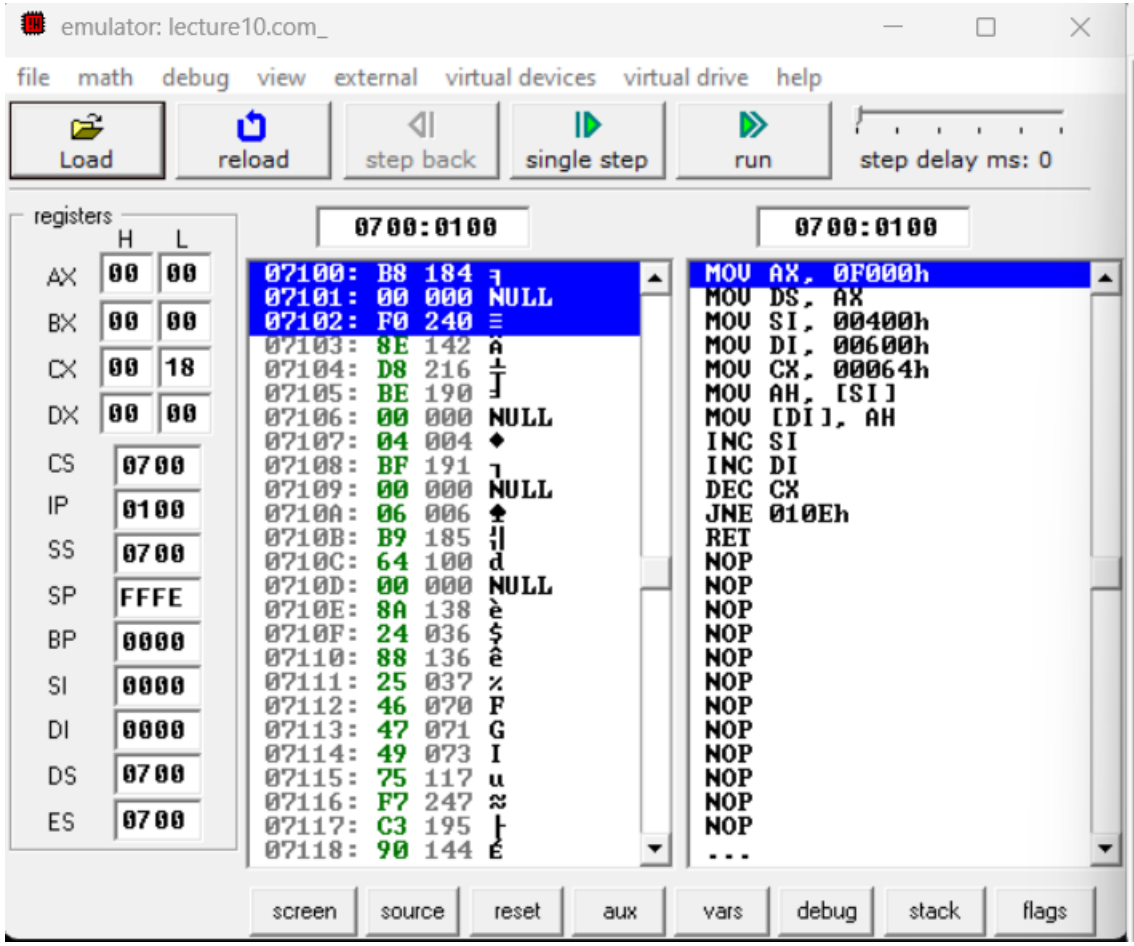
EXAMPLE

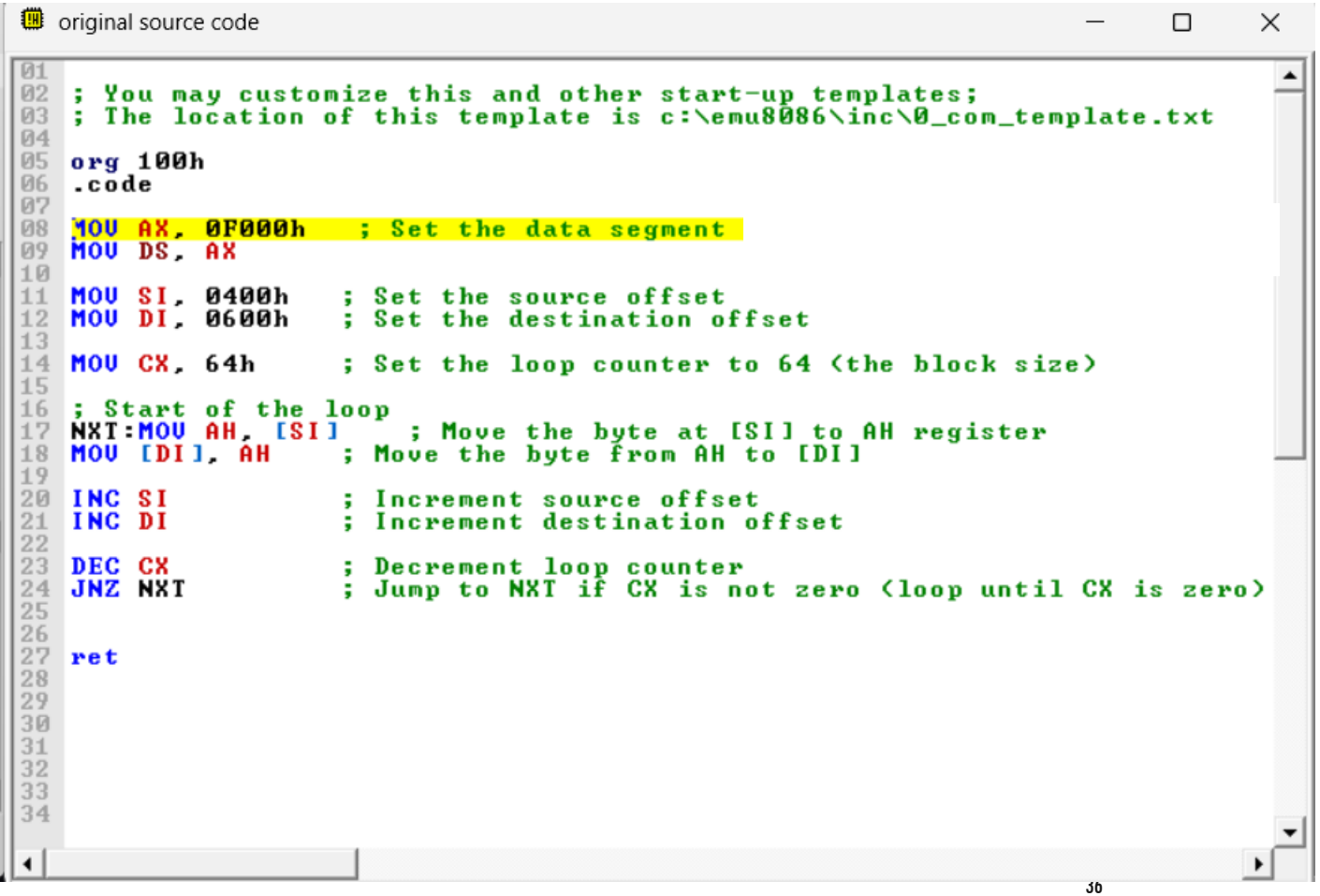
Ex: Write a program to move a block of 100 consecutive bytes of data starting at offset address 400H in memory to another block of memory locations starting at offset address 600H. Assume both block at the same data segment F000H.

```
MOV AX, 0F000H
MOV DS, AX
MOV SI, 0400H
MOV DI, 0600H
MOV CX, 64H  →  64 Hexadecimal = 100 Decimal

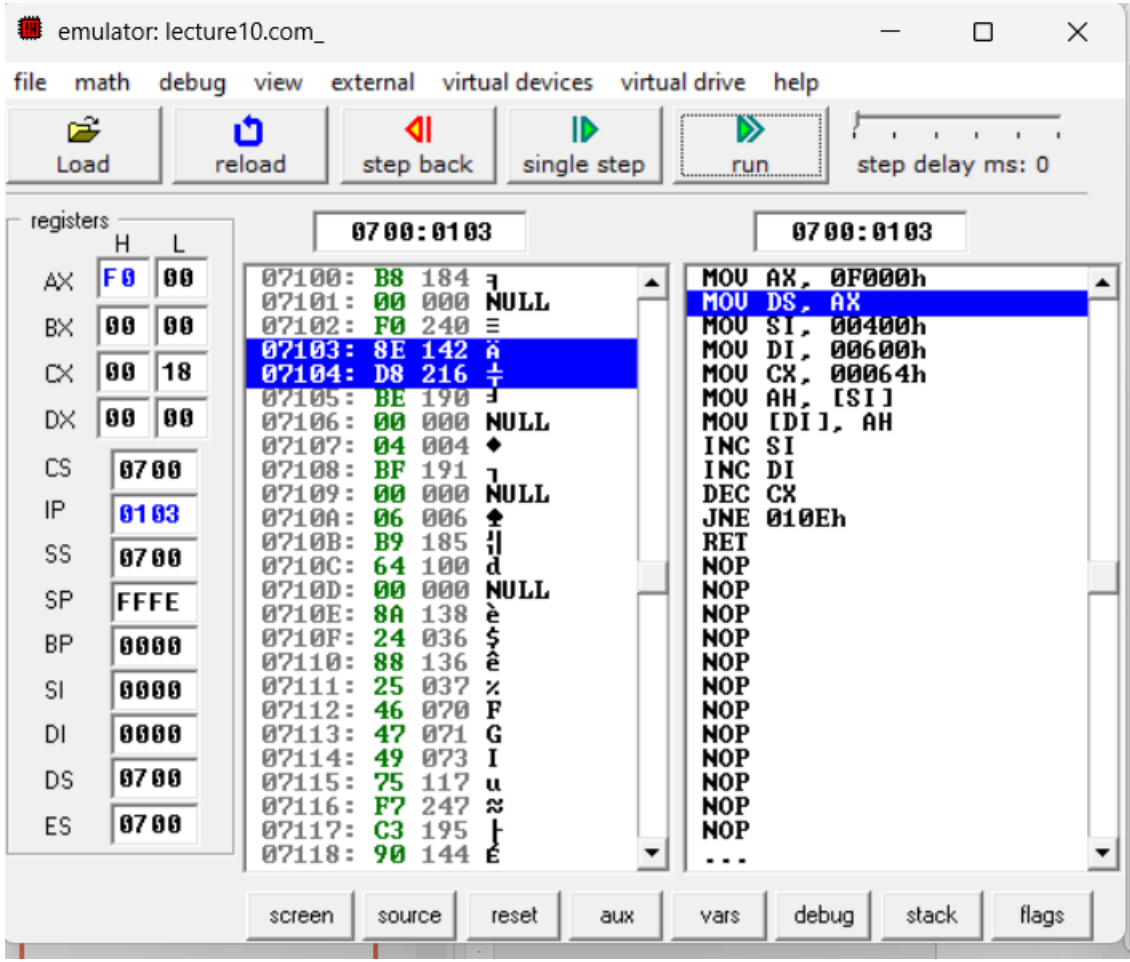
LableX: MOV AH, [SI]
        MOV [DI], AH
        INC SI
        INC DI
        DEC CX
        JNZ LableX
        RET        →  End of program
```

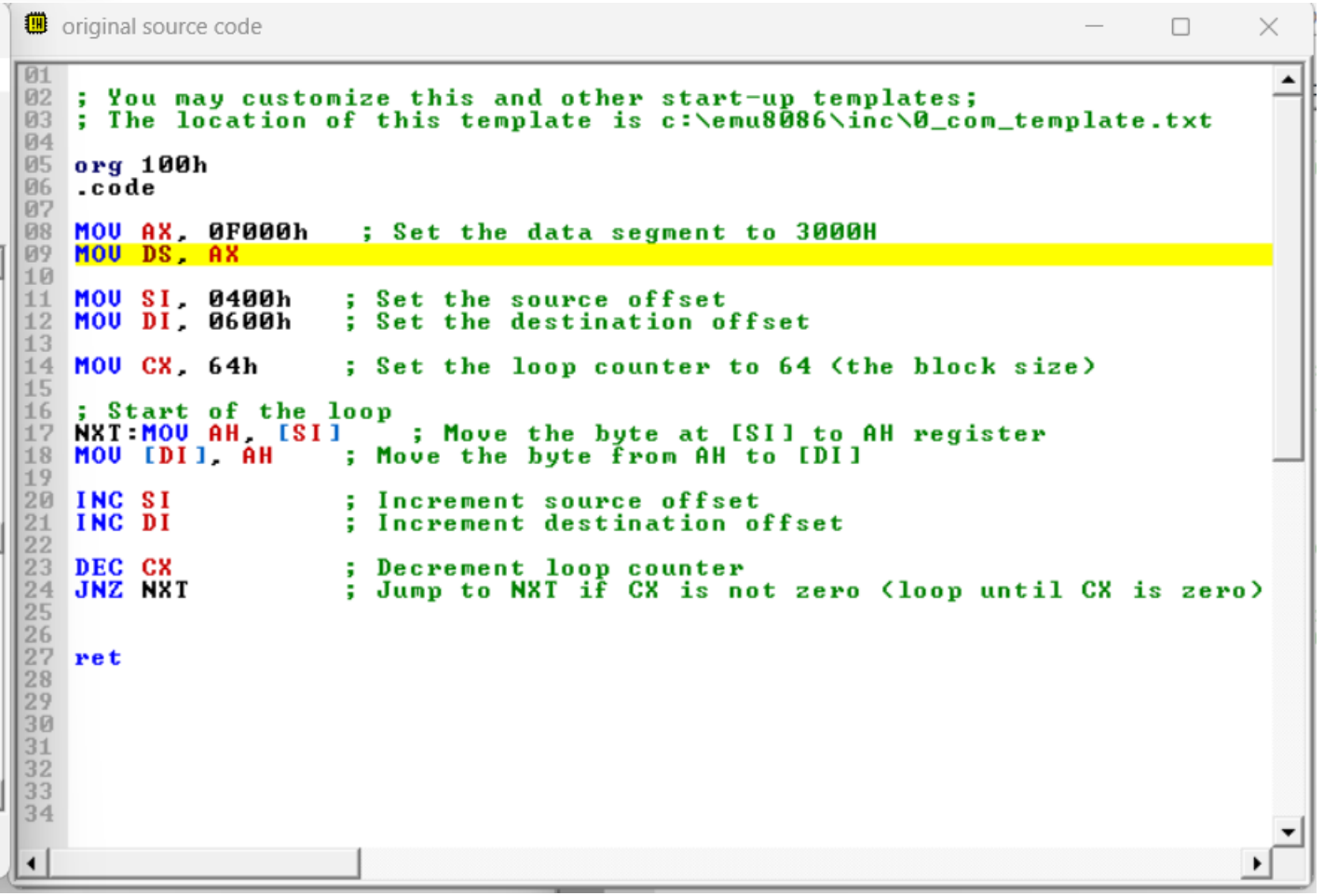
Observe the CS, registers, IP before executing the first instruction



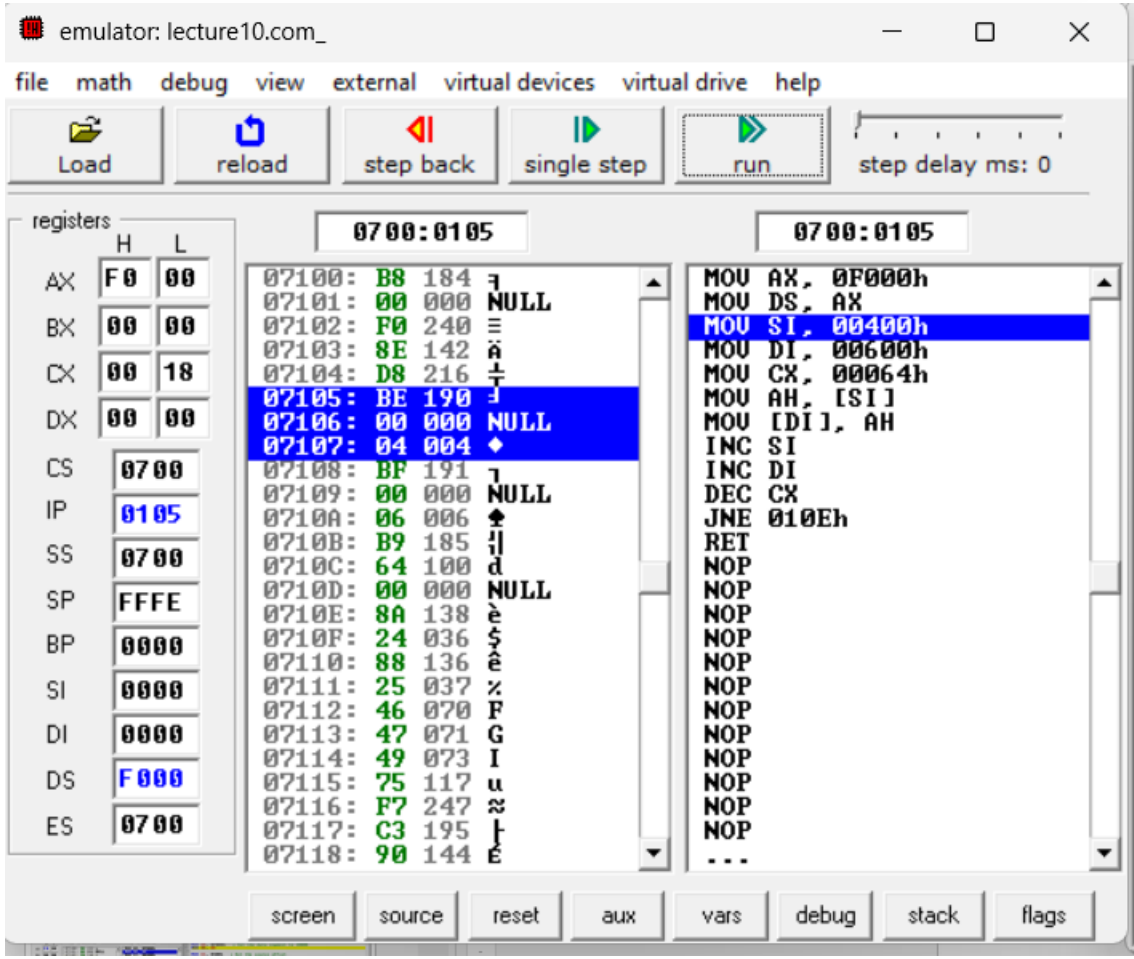


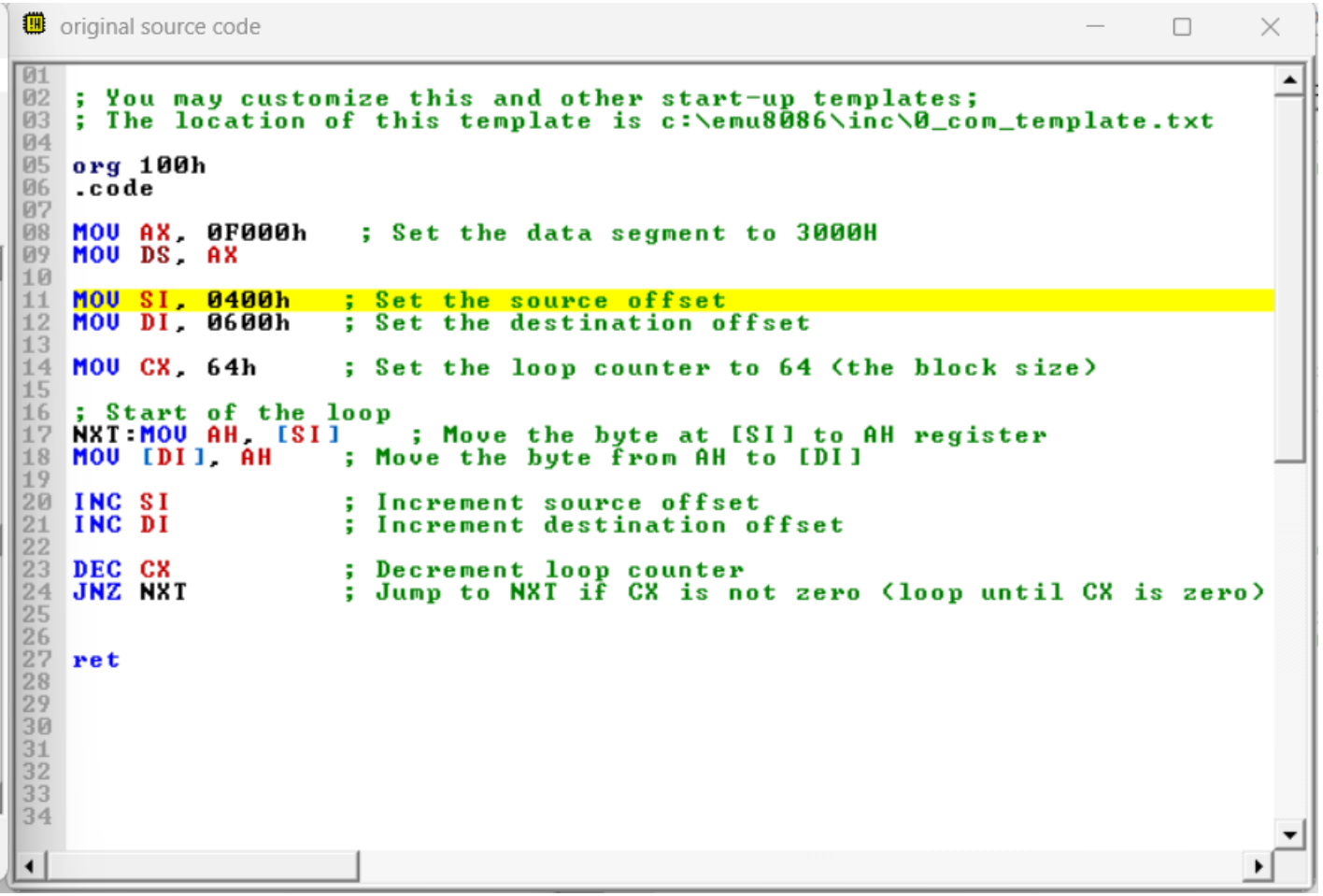
After Executing: MOV AX, 0F000h
Observe AX, IP



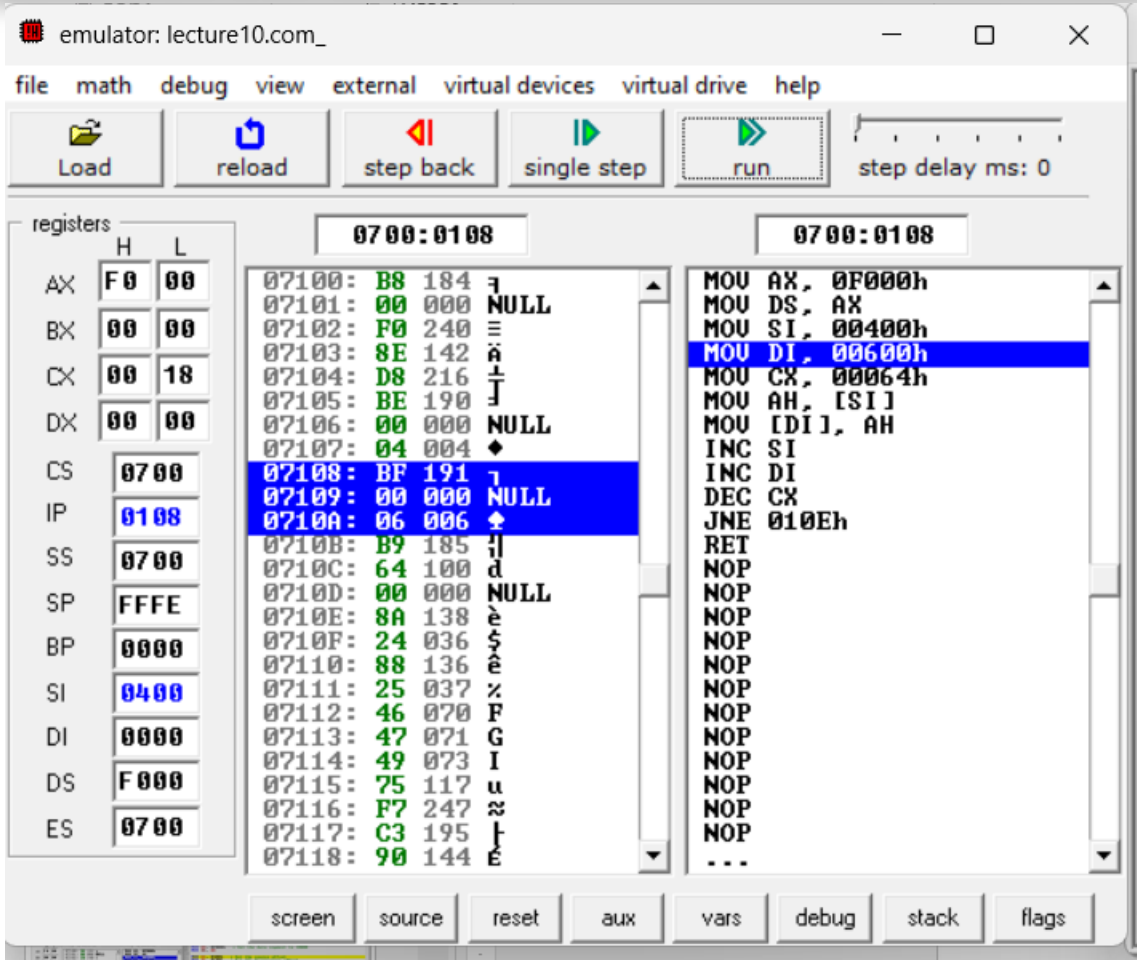


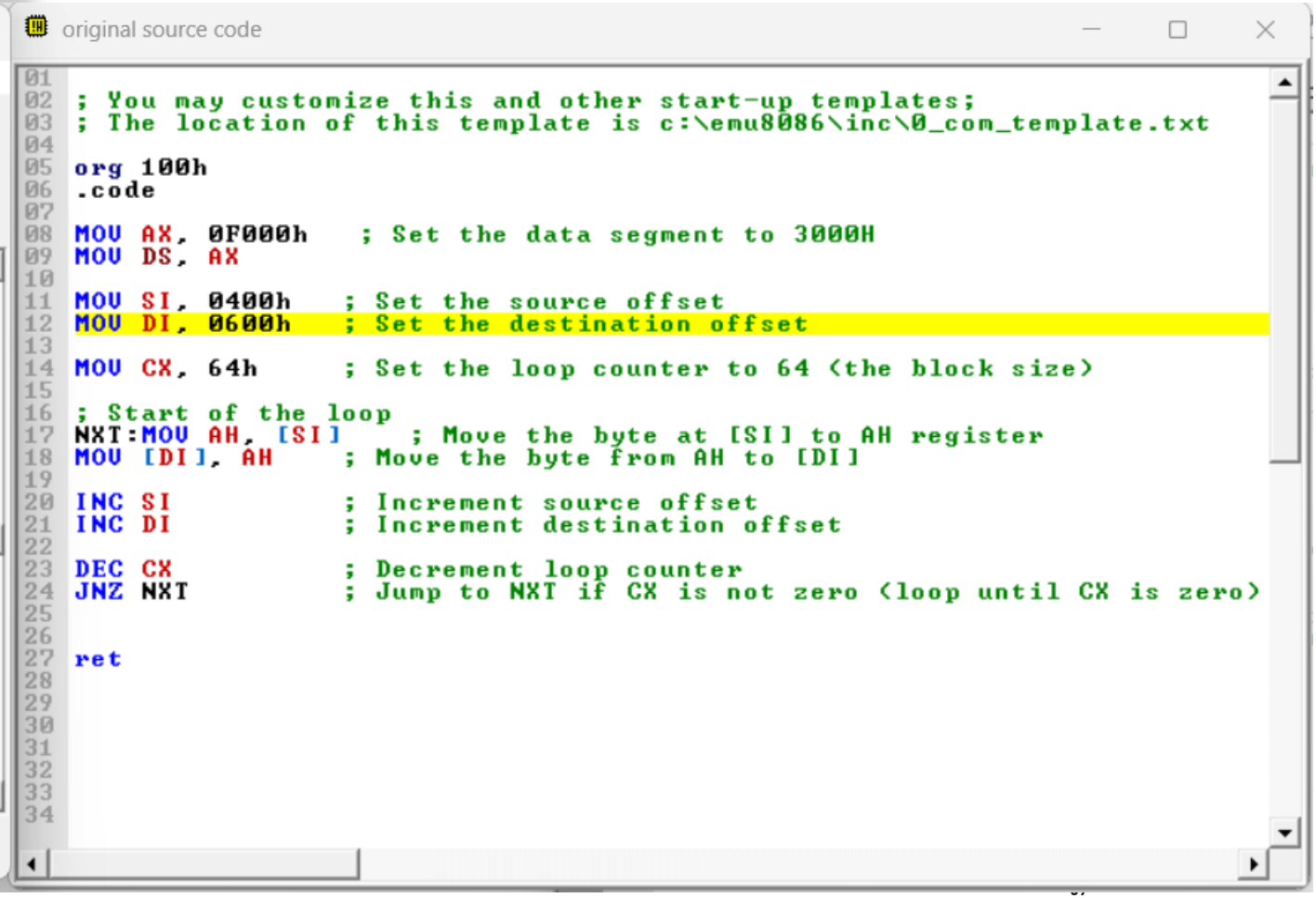
After Executing: MOV DS, AX
Observe AX, IP, DS



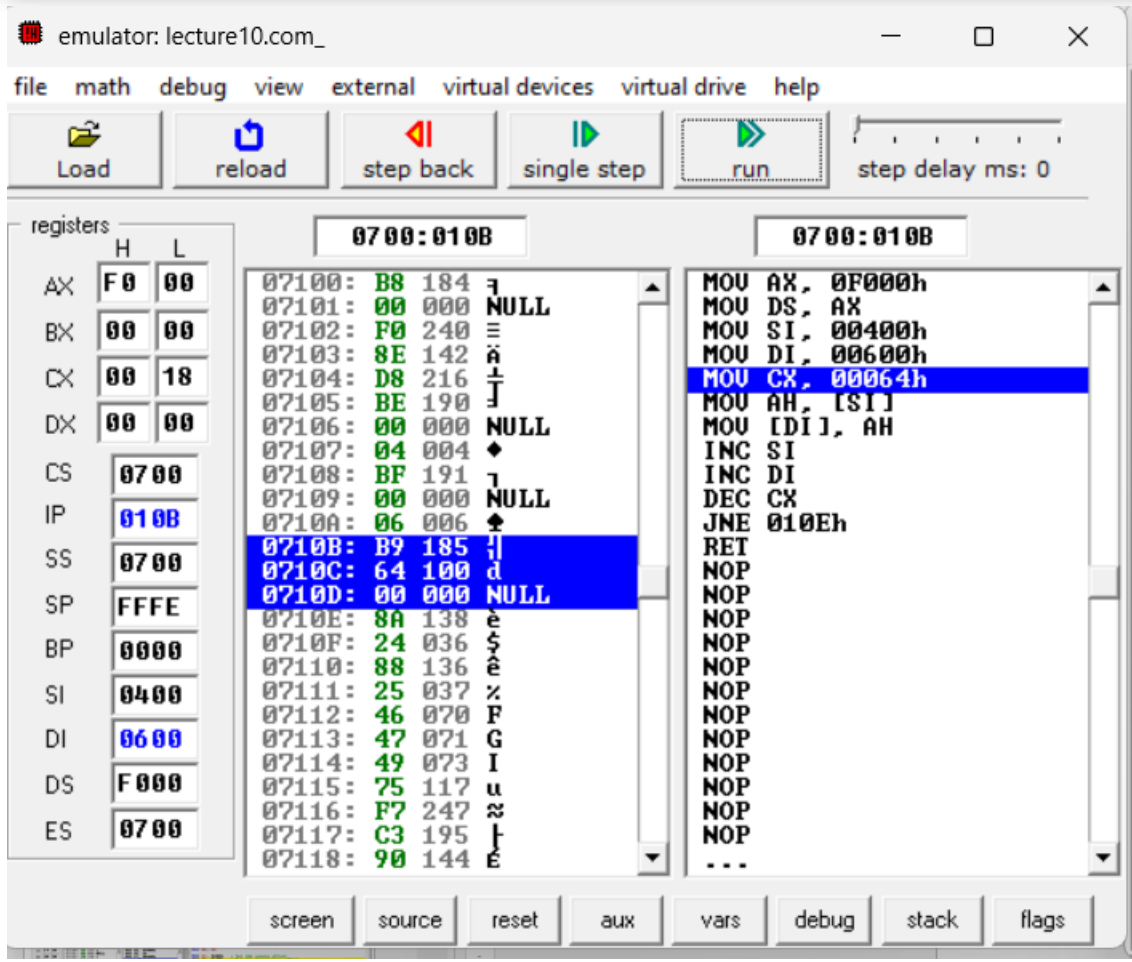


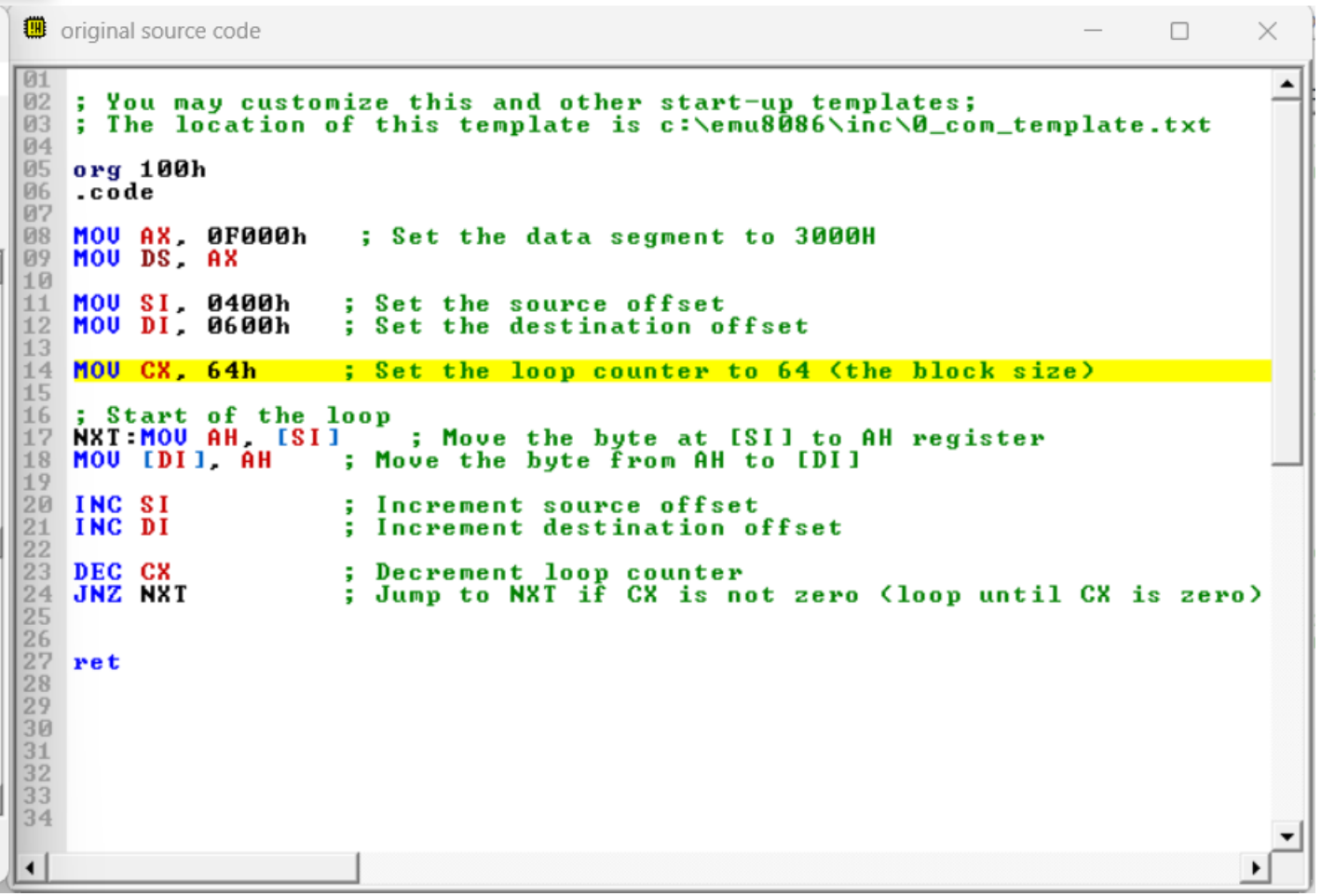
After Executing: MOV SI, 0400h
Observe SI, IP



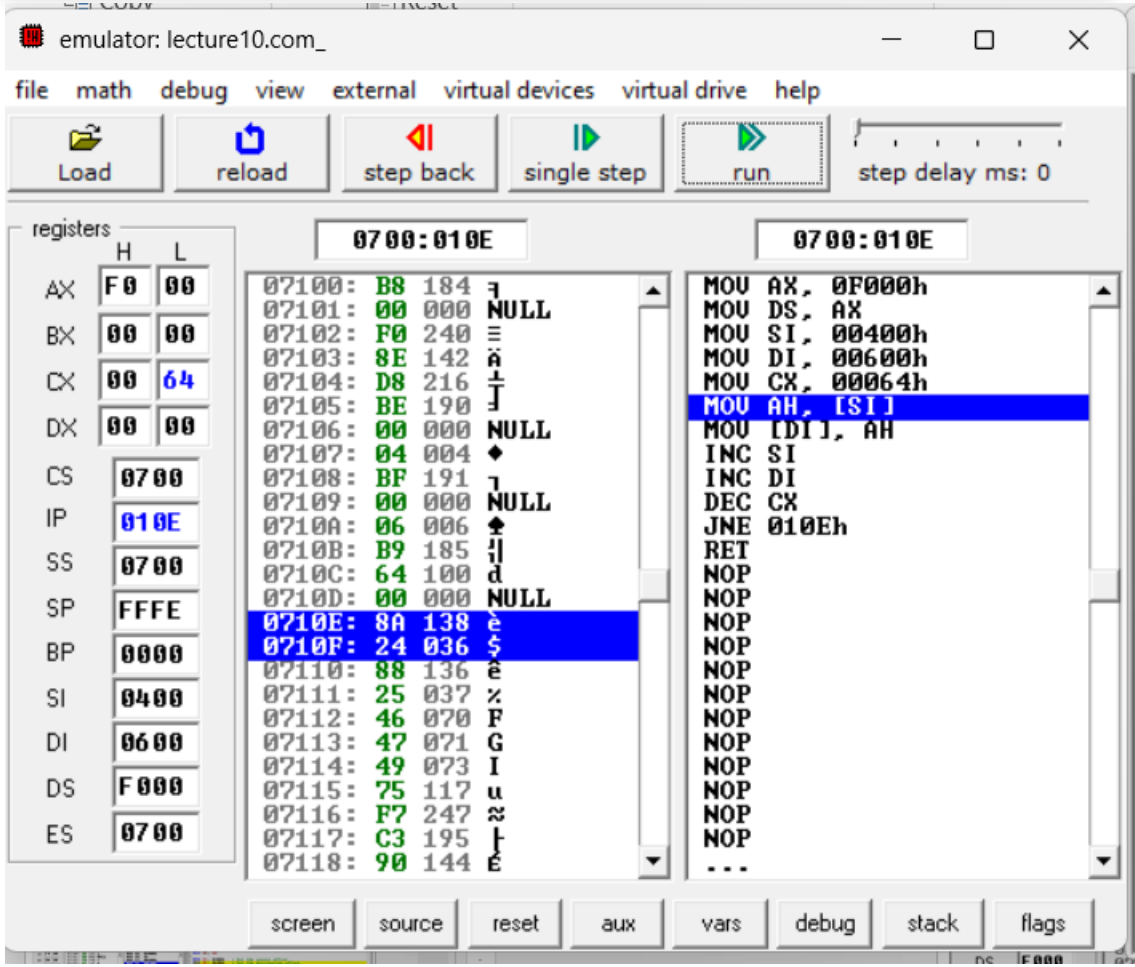


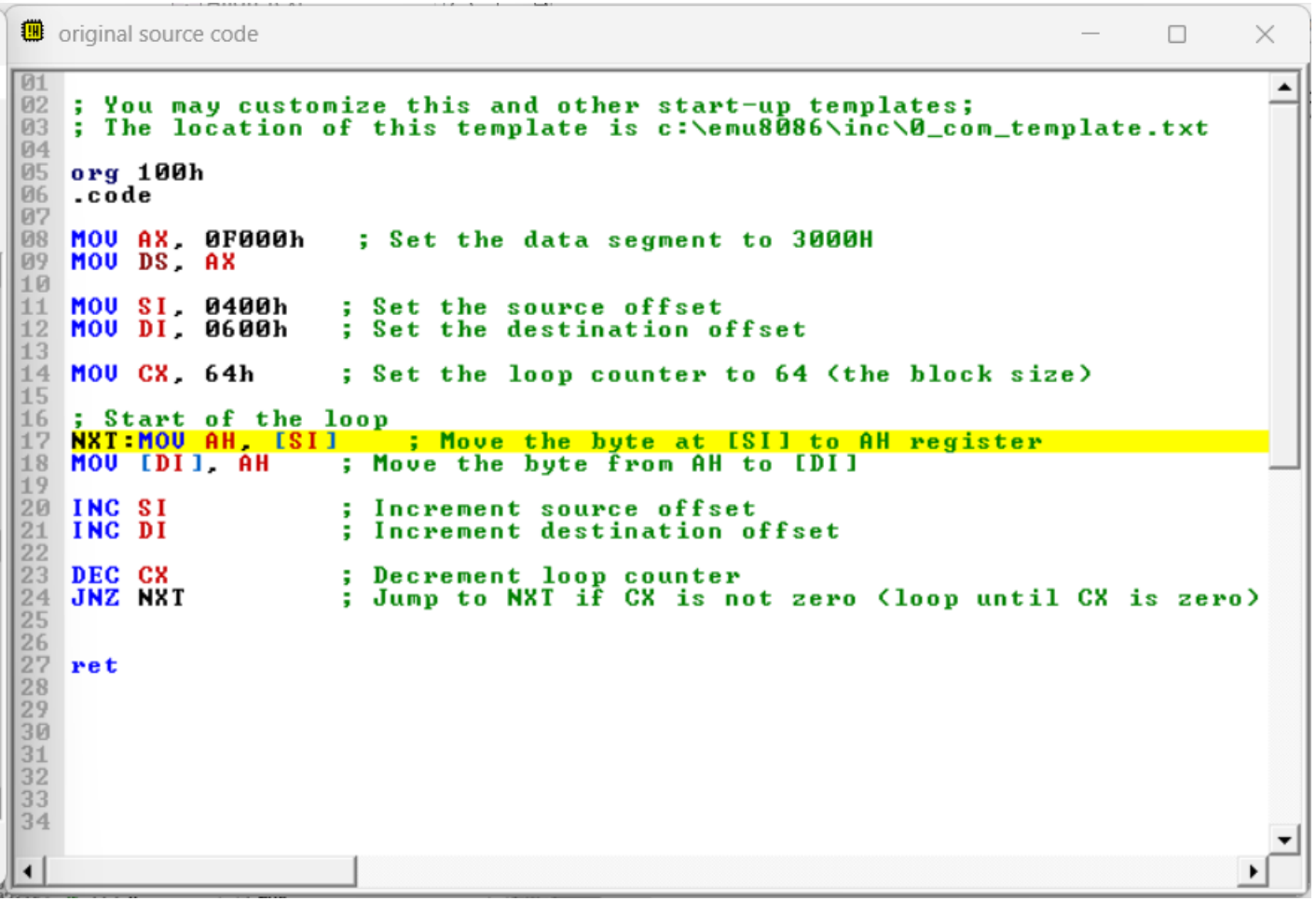
After Executing: MOV DI, 0600h
Observe DI, IP



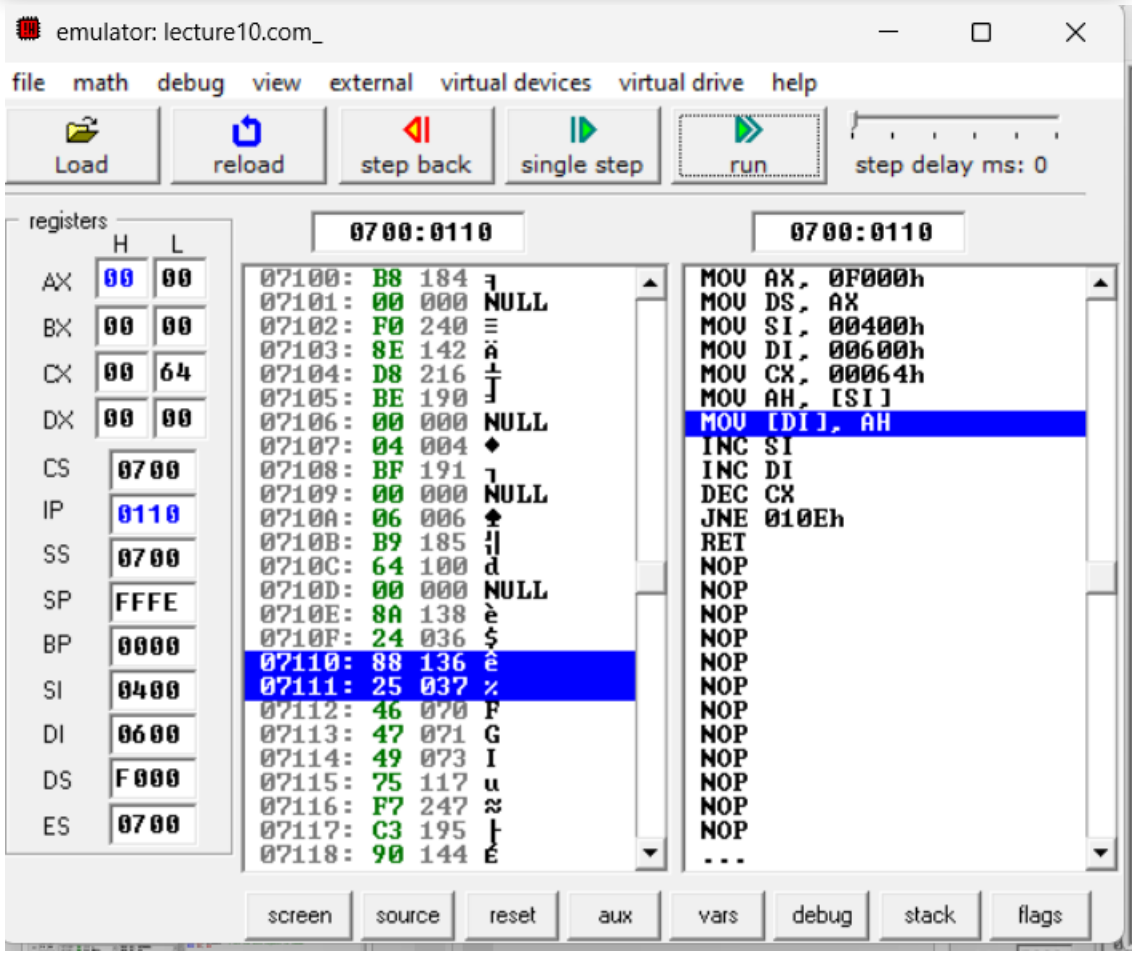


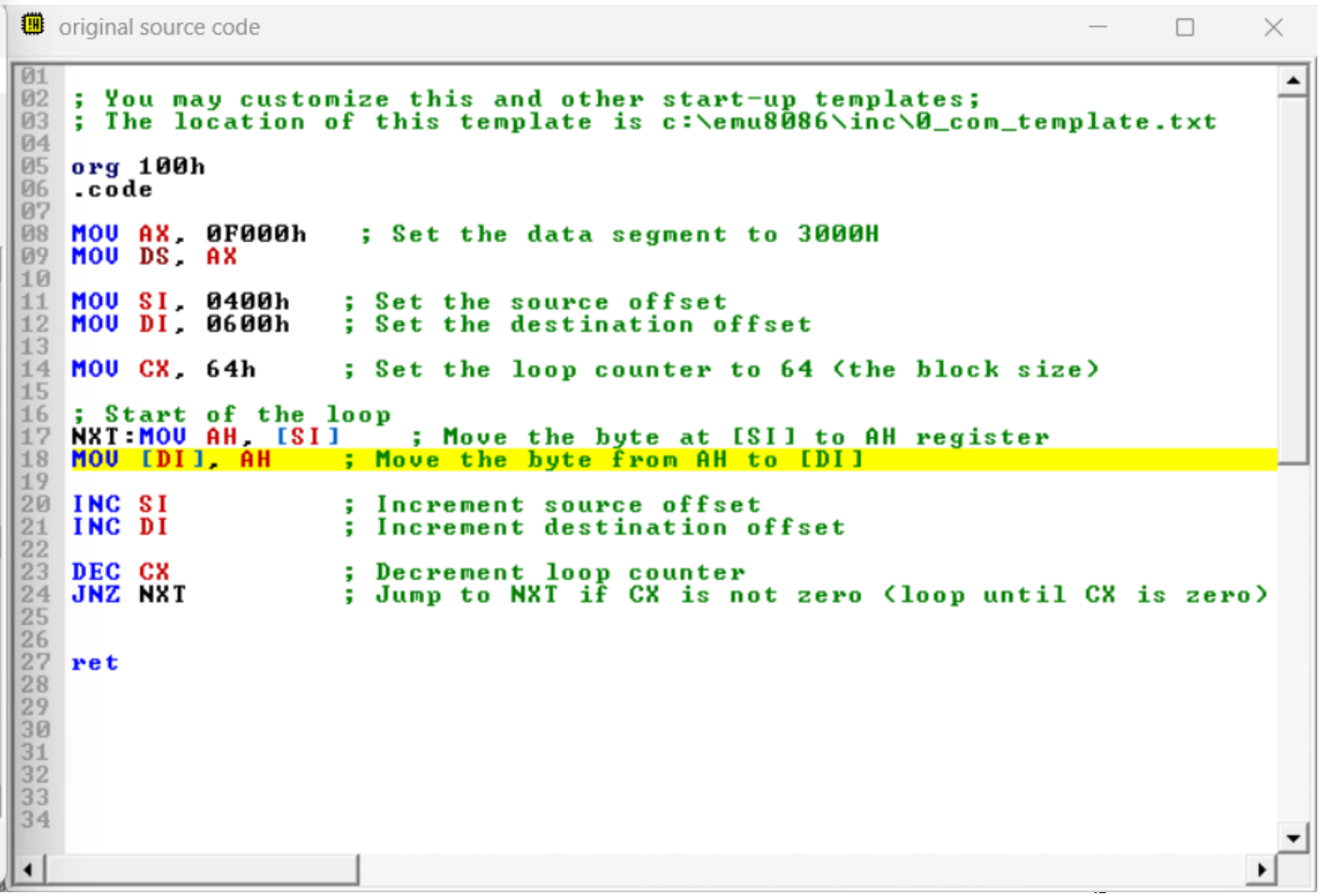
After Executing: MOV CX, 64h
Observe CX, IP



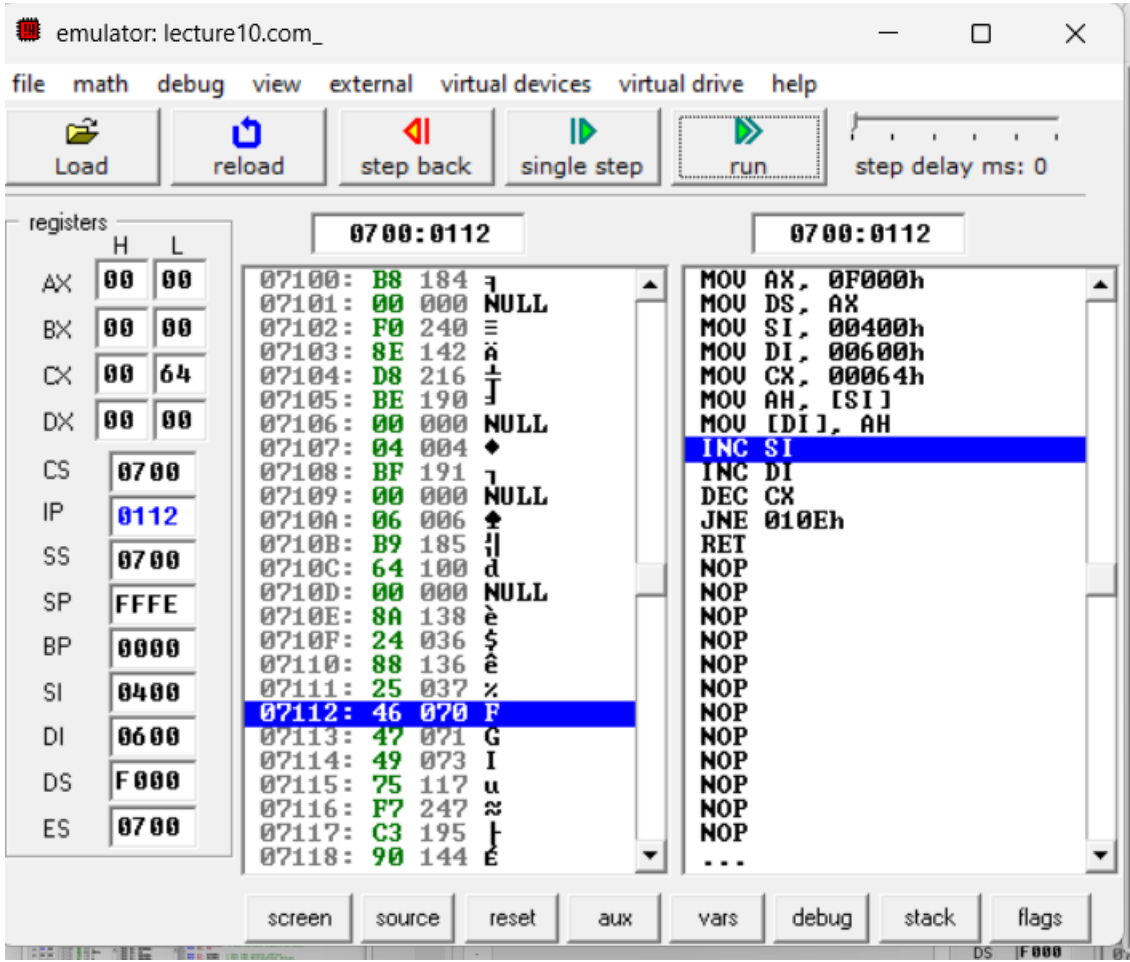


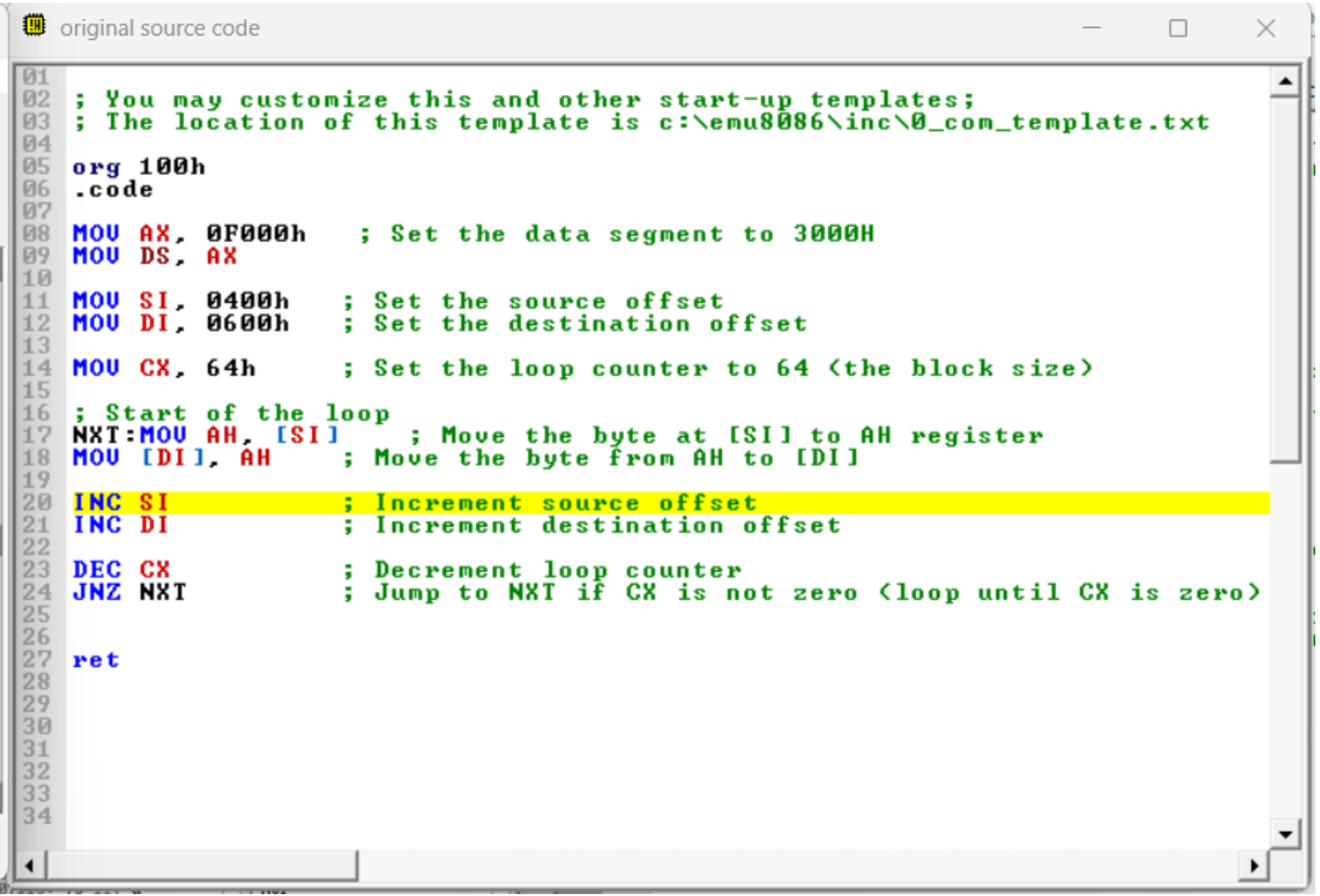
After Executing: MOV AH, [SI]
Observe AX, IP





After Executing: MOV [DI], AH
Observe AX, IP





After Executing: INC SI
Observe SI, IP



Why is the IP incremented by only 1 in this step ?

emulator: lecture10.com_

file math debug view external virtual devices virtual drive help

Load | reload | step back | single step | run | step delay ms: 0

registers

	H	L
AX	00	00
BX	00	00
CX	00	64
DX	00	00
CS	0700	
IP	0113	
SS	0700	
SP	FFFE	
BP	0000	
SI	0401	
DI	0600	
DS	F000	
ES	0700	

0700:0113

```
07100: B8 184 ╕
07101: 00 000 NULL
07102: F0 240 ≡
07103: 8E 142 Ä
07104: D8 216 ╪
07105: BE 190 ╛
07106: 00 000 NULL
07107: 04 004 ♦
07108: BF 191 ┐
07109: 00 000 NULL
0710A: 06 006 ♠
0710B: B9 185 ╣
0710C: 64 100 d
0710D: 00 000 NULL
0710E: 8A 138 è
0710F: 24 036 $
07110: 88 136 ê
07111: 25 037 %
07112: 46 070 F
07113: 47 071 G
07114: 49 073 I
07115: 75 117 u
07116: F7 247 ≈
07117: C3 195 ├
07118: 90 144 É
```

0700:0113

```
MOV AX, 0F000h
MOV DS, AX
MOV SI, 00400h
MOV DI, 00600h
MOV CX, 00064h
MOV AH, [SI]
MOV [DI], AH
INC SI
INC DI
DEC CX
JNE 010Eh
RET
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
```

screen | source | reset | aux | vars | debug | stack | flags

original source code

```
; You may customize this and other start-up templates;
; The location of this template is c:\emu8086\inc\0_com_template.txt

org 100h
.code

MOV AX, 0F000h    ; Set the data segment to 3000H
MOV DS, AX

MOV SI, 0400h     ; Set the source offset
MOV DI, 0600h     ; Set the destination offset

MOV CX, 64h       ; Set the loop counter to 64 (the block size)

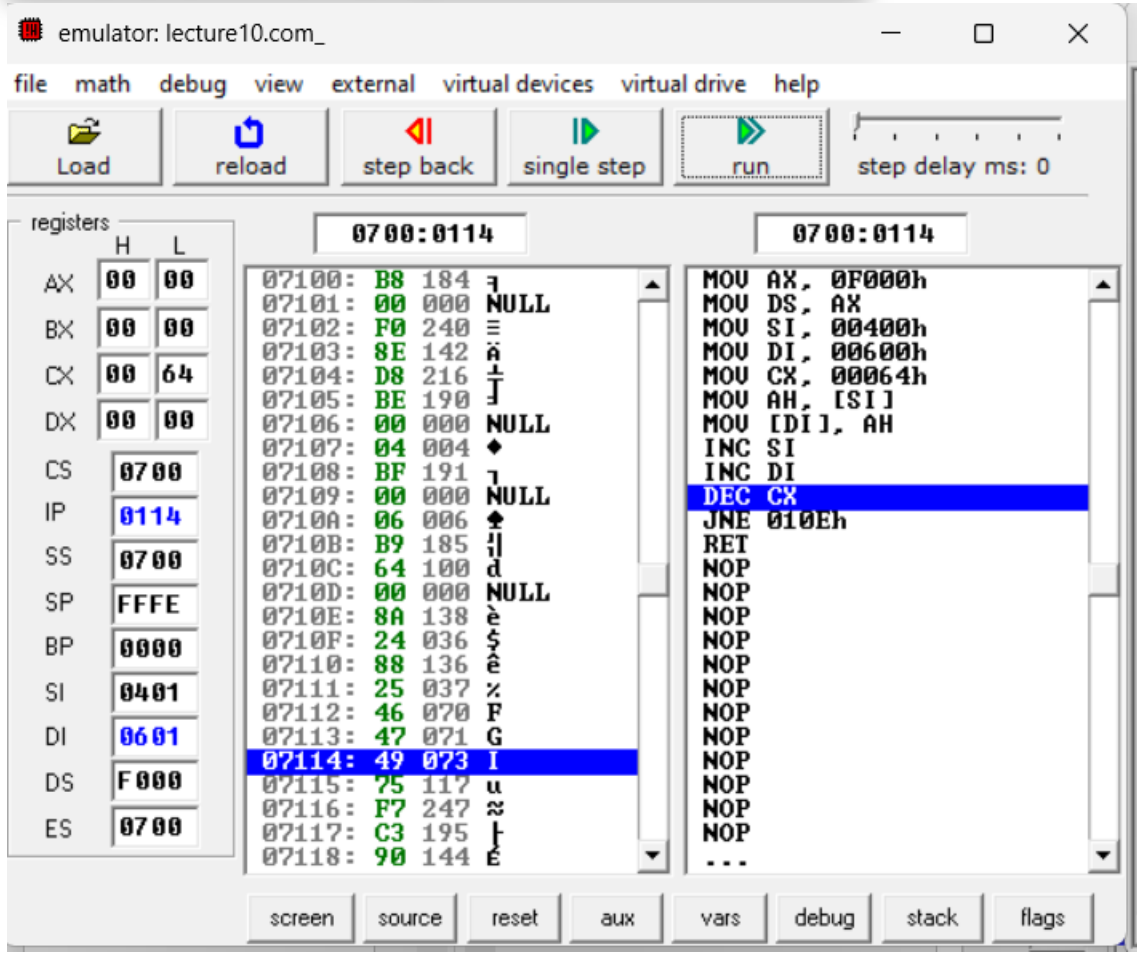
; Start of the loop
NXT:MOV AH, [SI]     ; Move the byte at [SI] to AH register
MOV [DI], AH      ; Move the byte from AH to [DI]

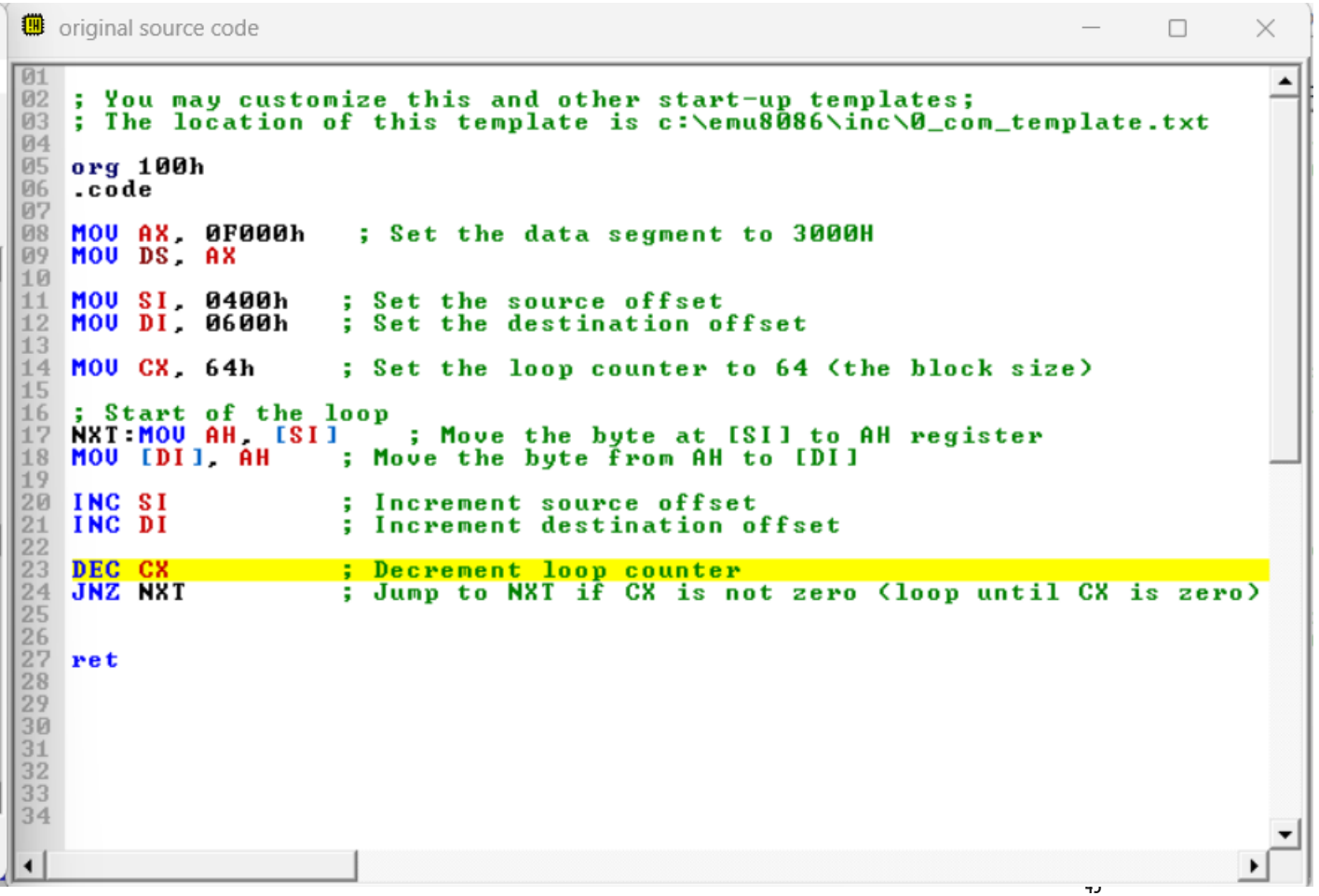
INC SI            ; Increment source offset
INC DI            ; Increment destination offset

DEC CX            ; Decrement loop counter
JNZ NXT           ; Jump to NXT if CX is not zero (loop until CX is zero)

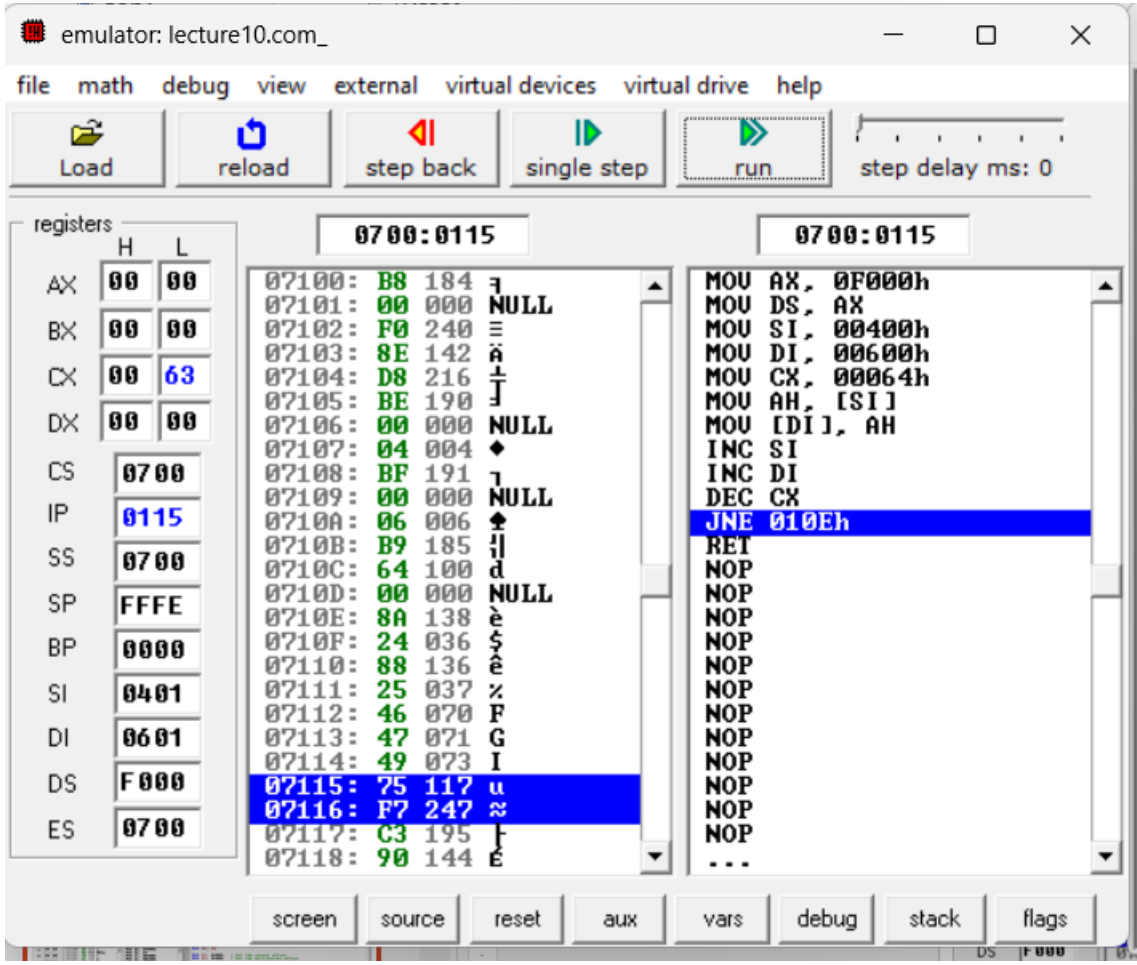

ret
```

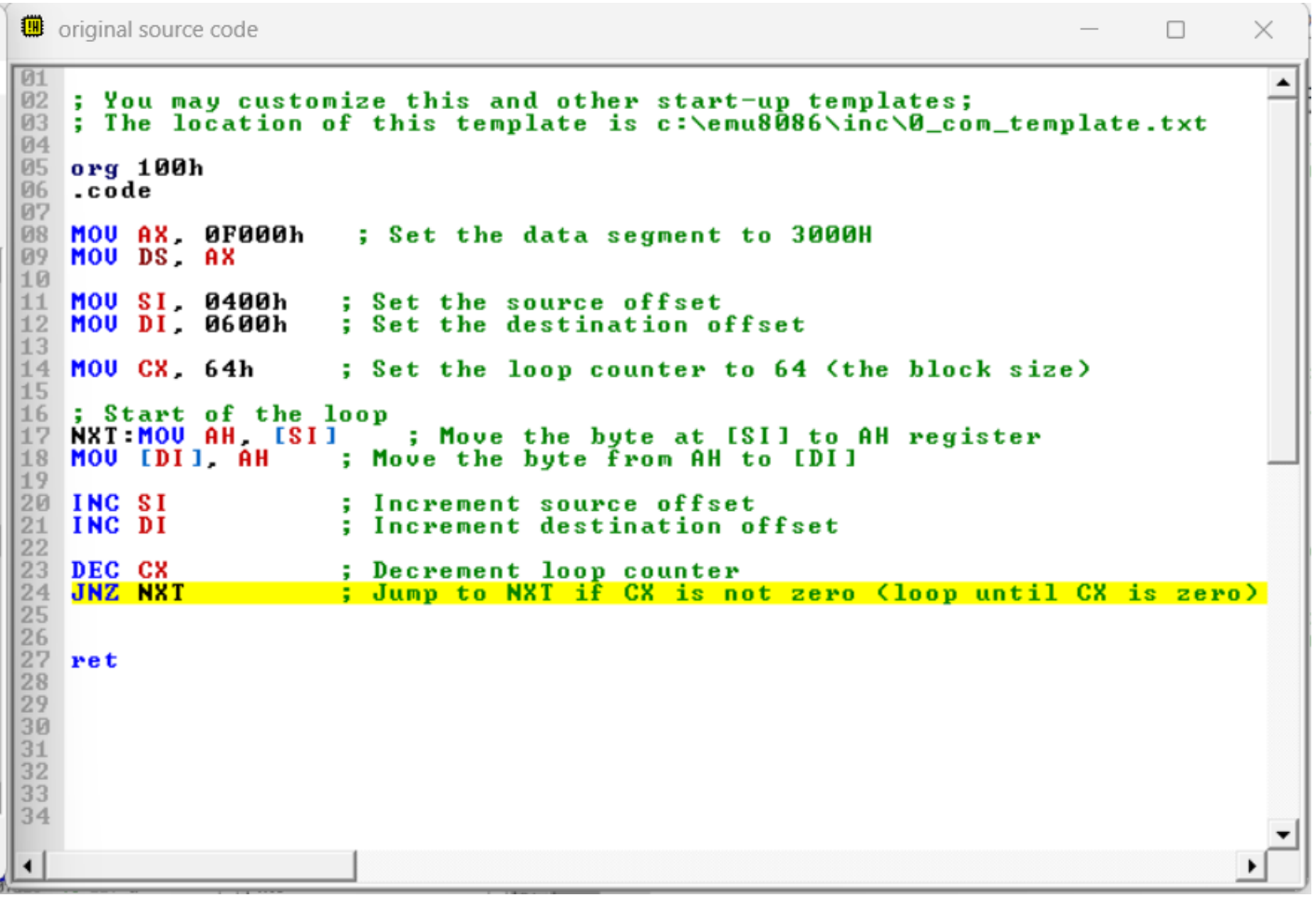
After Executing: INC DI
Observe DI, IP





After Executing: DEC CX
Observe CX, IP





Complete running your code step by step and

Observe the changes occurring to the registers

LOOP

- A combination of a **decrement CX** and the **JNZ conditional jump**.
- In 8086 - 80286 LOOP decrements CX.
  - if **CX != 0**, it jumps to the address indicated by the label
  - If **CX becomes 0**, the next sequential instruction executes
- In 80386 and above, LOOP decrements either CX or ECX, depending upon instruction mode.

LOOP

```
MOV    CX, 0064H        ; Initialize counter with 100(10)
BACK : _ _ _ _ _ _      ; This section is
       _ _ _ _ _ _      ; executed 100
       _ _ _ _ _ _      ; times i.e., until
       _ _ _ _ _ _      ; CX = 0
       LOOP BACK
```

LOOP Structure

Ref: https://www.electronicsmind.com/loop-instructions-of-8086-microprocessor/

EXAMPLE ON LOOPS

EXAMPLE

Write the final values after executing the code for AX, BX and CX. Show the changes in the memory.

	H	L
AX		
BX		
CX		

```
CS: 0200, DS: 0100
        MOV  AL,0
        MOV CL,04H
        MOV BX,0000H

AGAIN:  MOV [BX],AL
        INC AL
        INC BX
        LOOP AGAIN
        ret
```

Physical Address	Value

Ref: S. Vidhyadharan, "Microprocessors and Interfaces: Lecture 5 – 8086 Addressing Modes and OP-Code: Examples," lecture notes, 2021–2022.

CONDITIONAL LOOPS

- LOOP instruction also has conditional forms: **LOOPE** and **LOOPNE**
- **LOOPE** (loop while equal) instruction jumps if CX != 0 while an equal condition (Z = 1) exists.
  - will exit loop if the condition is not equal or the CX register decrements to 0
- **LOOPNE** (loop while not equal) jumps if CX != 0 while a not-equal condition (Z = 0) exists.
  - will exit loop if the condition is equal or the CX register decrements to 0

CONDITIONAL LOOPS



Assume that you want to test if all of 200 memory locations starting at the offset of 1680H contain 55H

```
        mov cx, 200
        mov si, 167Fh
Back:   inc si
        cmp [si], 55h
        loope back
        ret
```

CONDITIONAL LOOPS

EXAMPLE

Find the first day that had a 90 degree Fahrenheit in 30 days with the values stored at offset 1200h

```
        mov cx, 30
        mov si, 11FFh
Back:   inc si
        cmp [si], 90
         loopne back
        ret
```

REFERENCE TEXTBOOK

[1] “80X86 IBM PC and Compatible Computers: Assembly Language, Design, and Interfacing” by Muhammad Ali Mazidi and Janice Gillispie Mazidi

[2] “The Intel microprocessors 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro processor, Pentium II, Pentium III, Pentium 4, and Core2 with 64-bit extensions: architecture, programming, and interfacing” by Barry B. Brey”

REFERENCES

[1] https://binaryupdates.com/interview-question-answers-on-8086-microprocessor/

[2] https://www.careerride.com/8086-interview-questions.aspx